



Table of contents

01 Introduction

03 Bridge and
Articulation Point

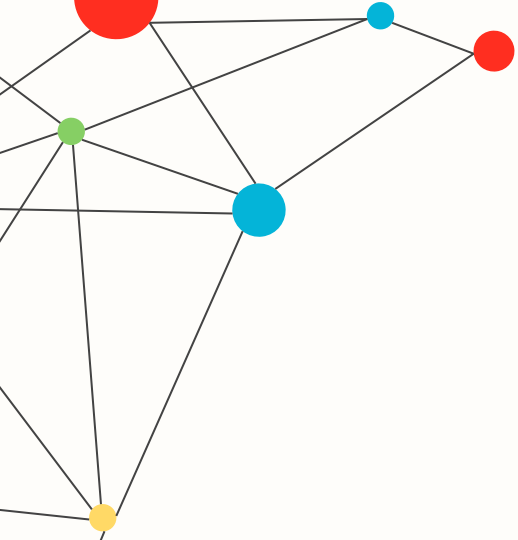
05 Applications

02 Basic definitions

04 Algorithm

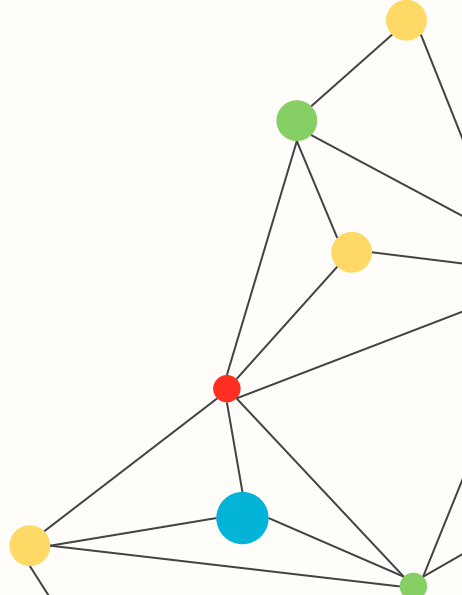
06 Exercises





01

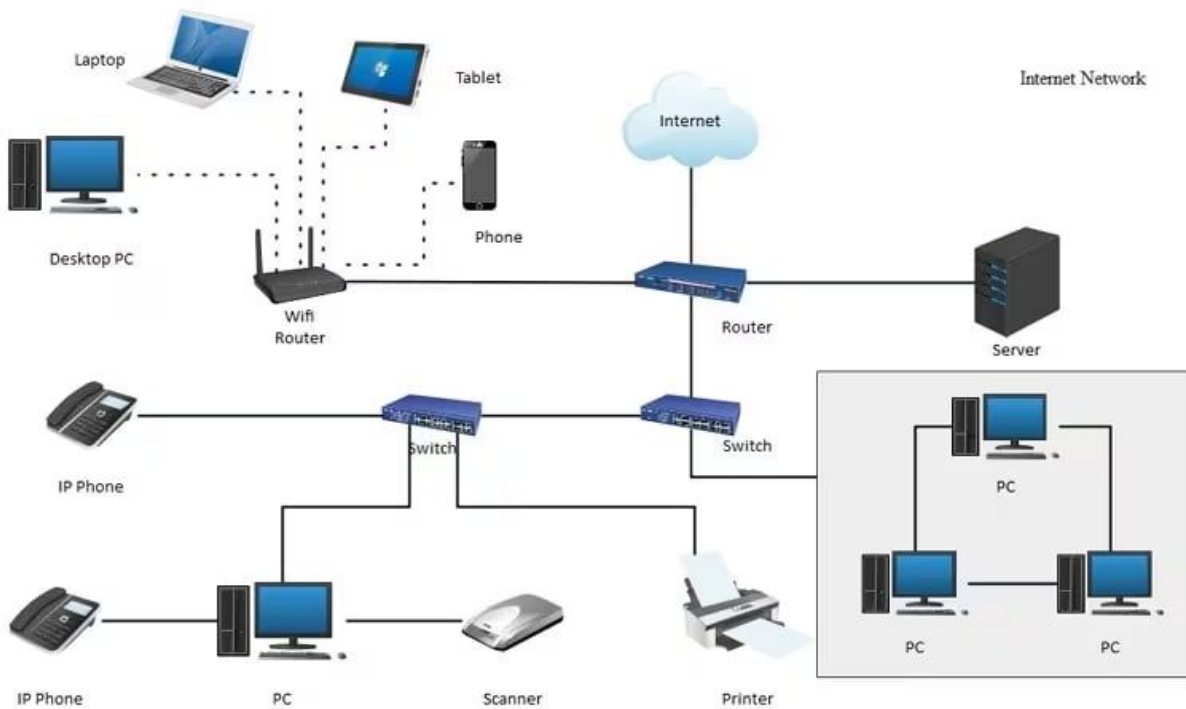
Introduction

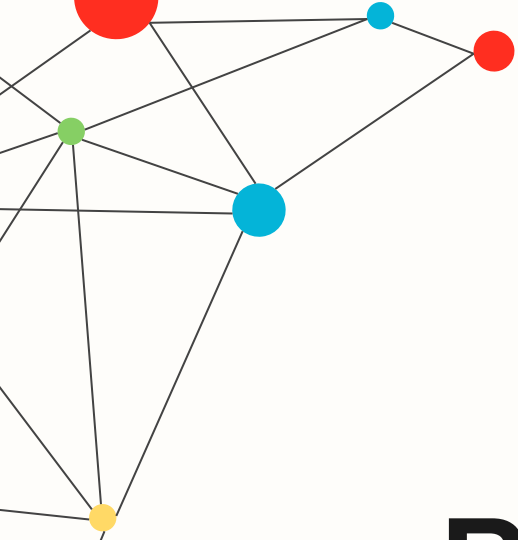


01. Introduction



01. Introduction

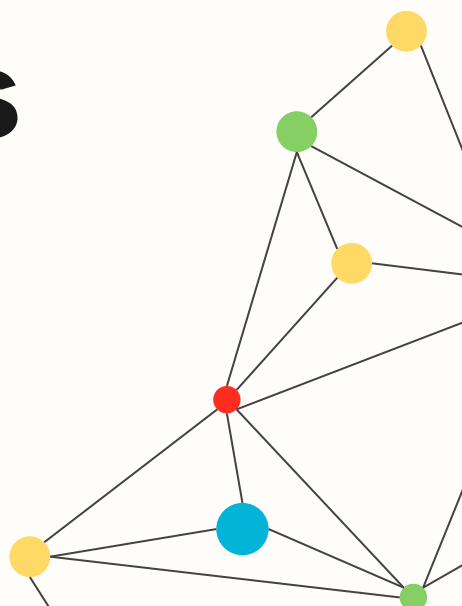




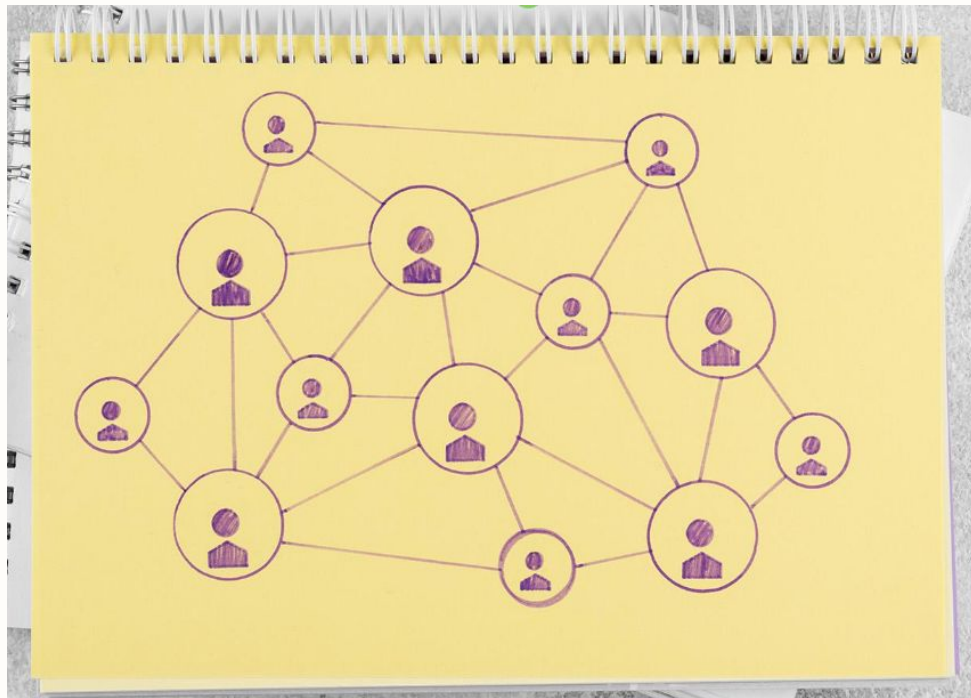
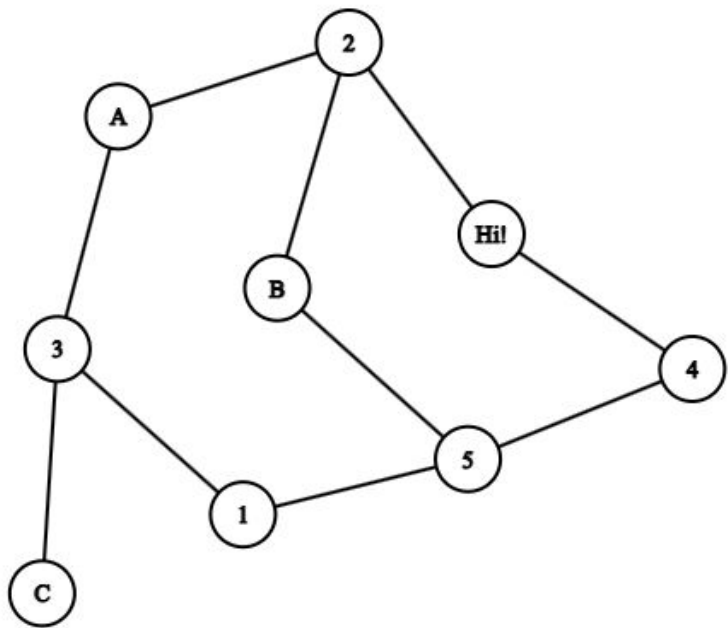
02

Basic Definitions

The founding concepts

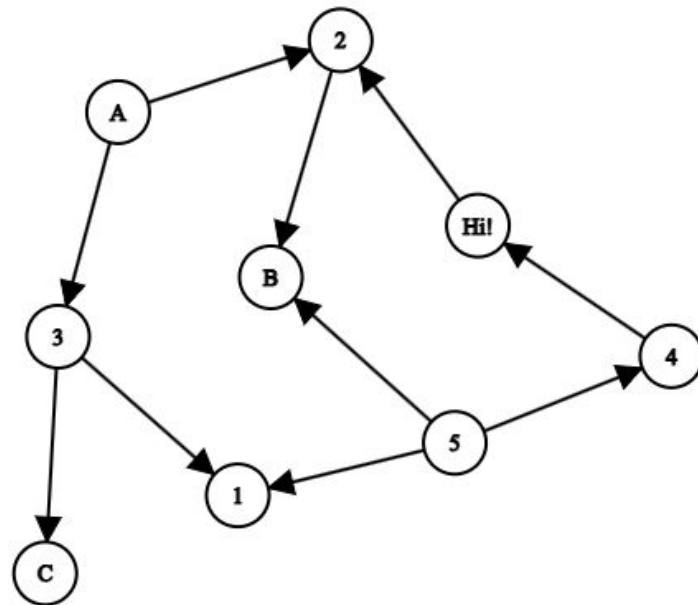
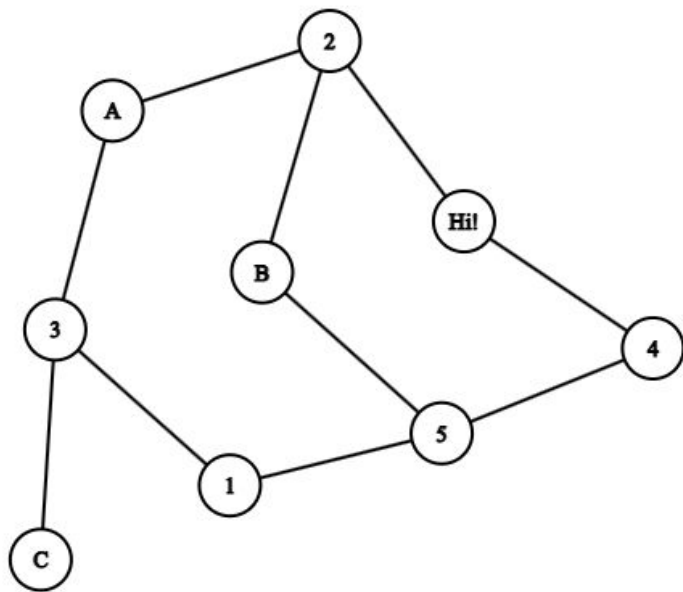


Graph



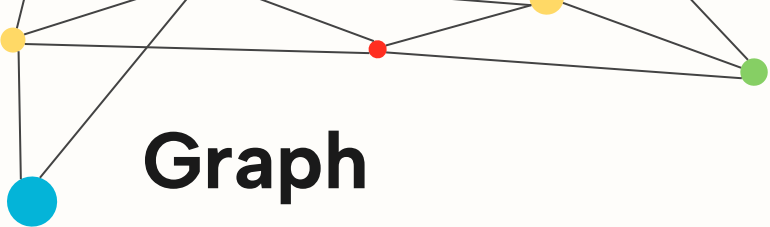
Graph

To focus on undirected!



• Undirected

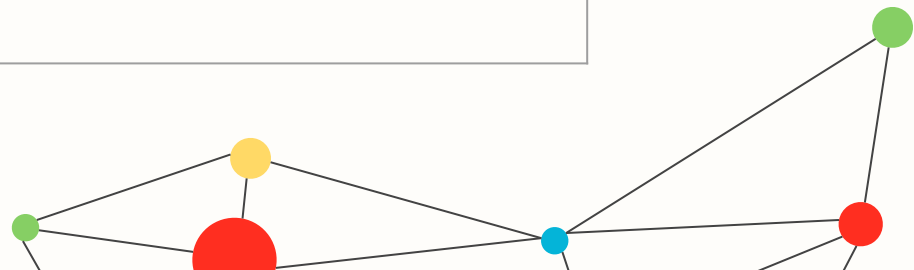
Directed



Graph

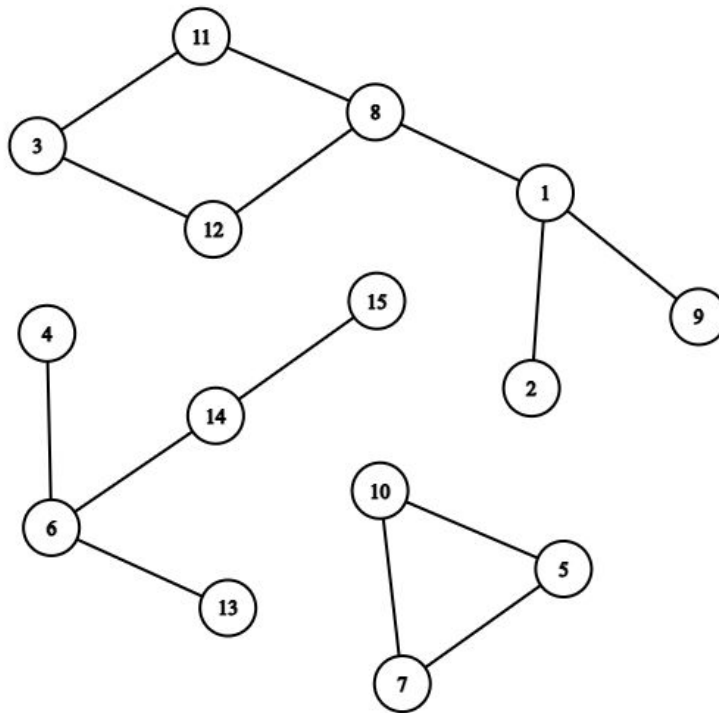
A **graph** is a structure that represents **relations** between **objects**

An undirected graph $G(V, E)$ is an ordered pair of two sets
⇒ A **set V of vertices** (nodes).
⇒ A **set E of edges** (links). Elements of E are unordered pair (u, v) where $u, v \in V$



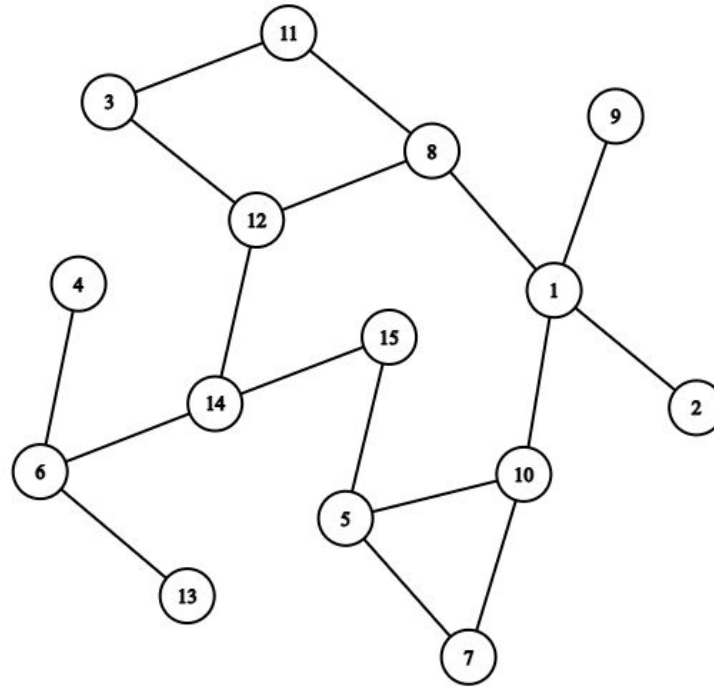
Connectivity

Two nodes are connected if there is some path between them



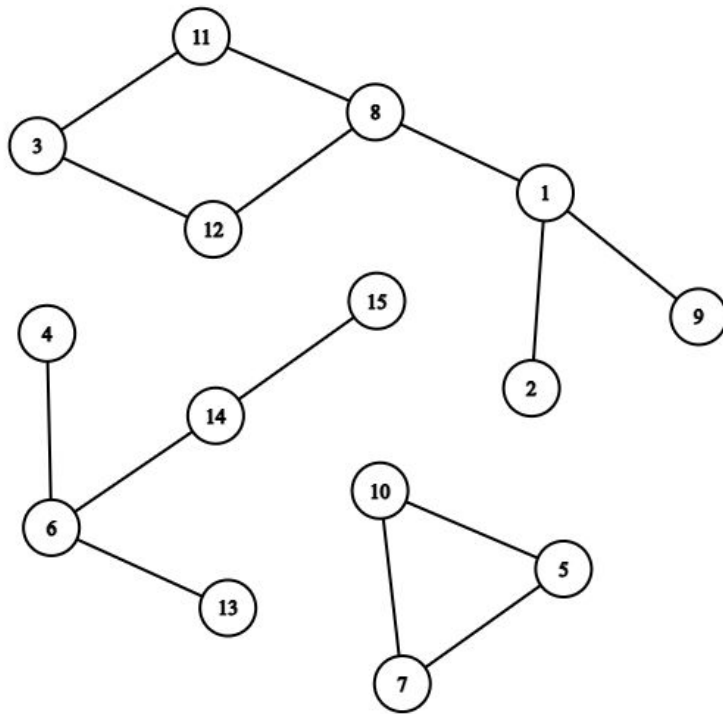
Connectivity

Graph is connected if any pair of nodes is connected

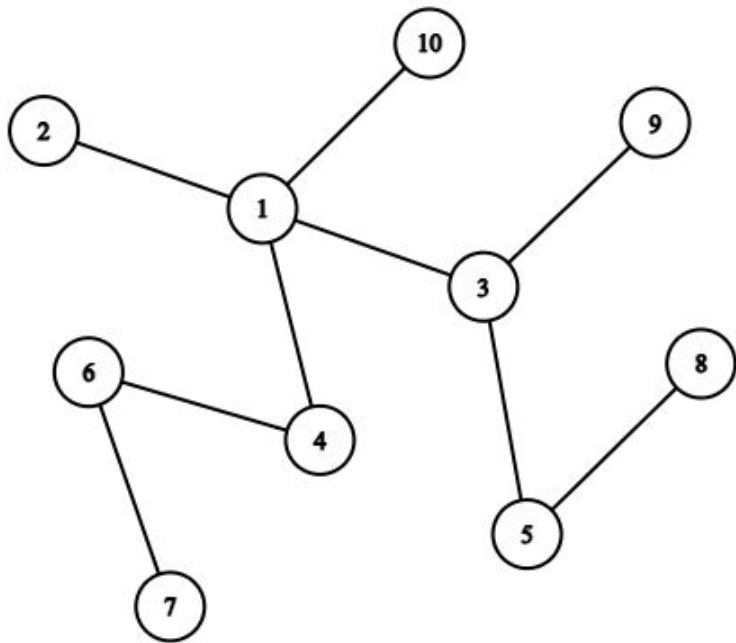


Connectivity

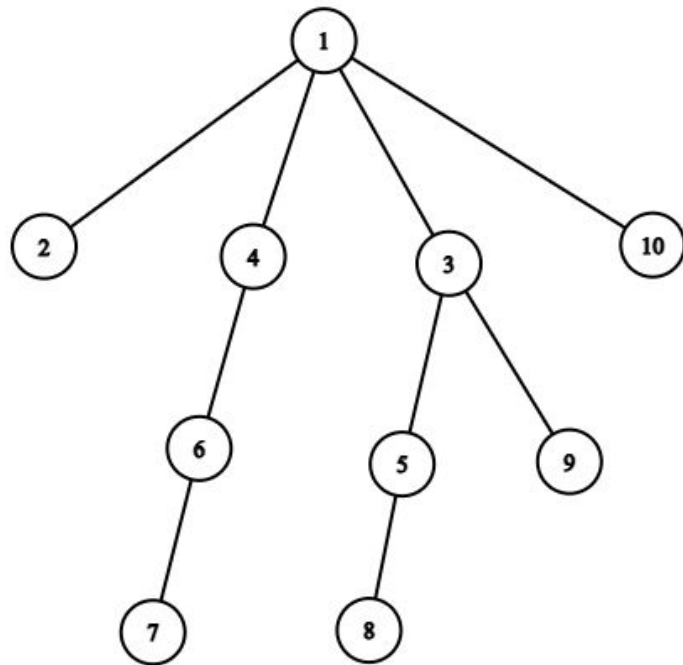
A **connected component** is a maximal connected subgraph.



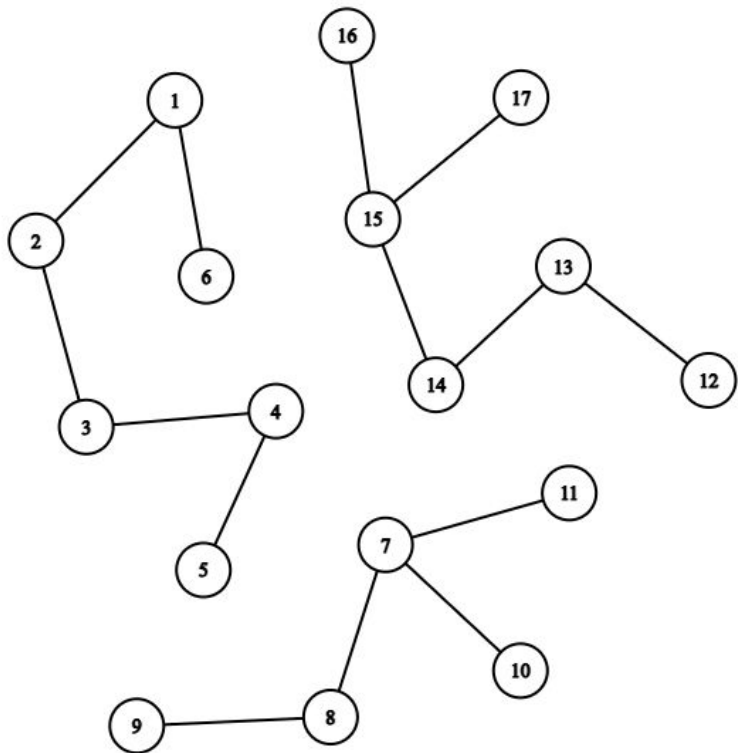
Tree and forest

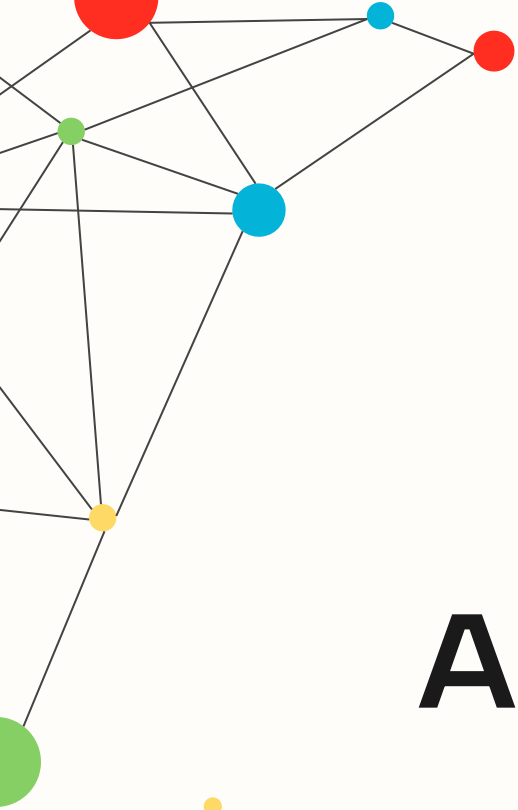


Rooted at 1



Tree and forest



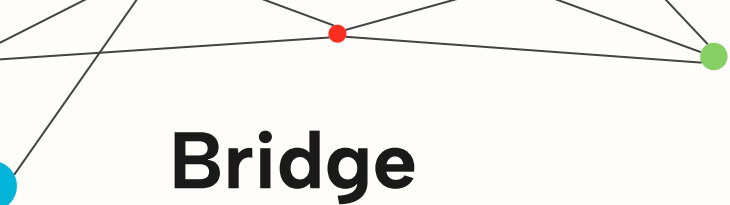


03

Bridge and Articulation Point

The what?



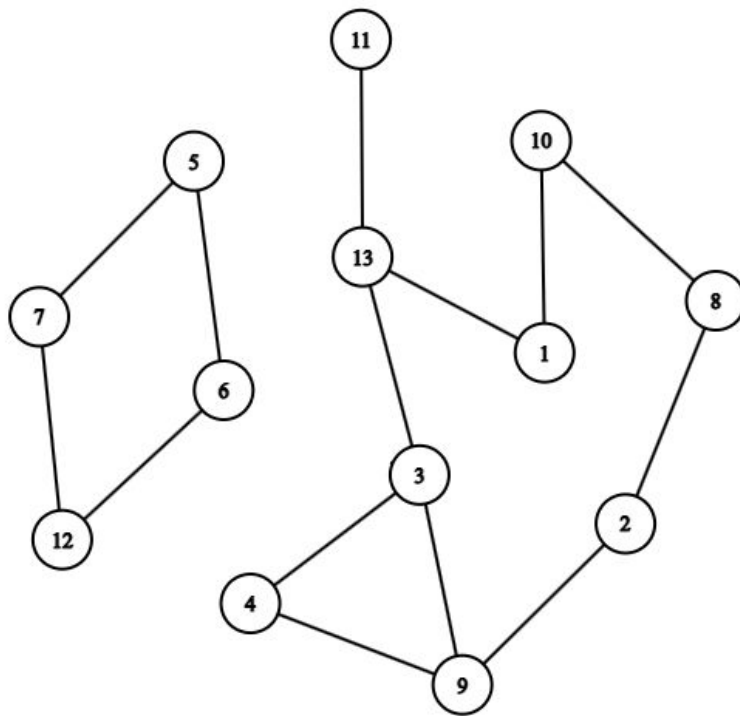


Bridge

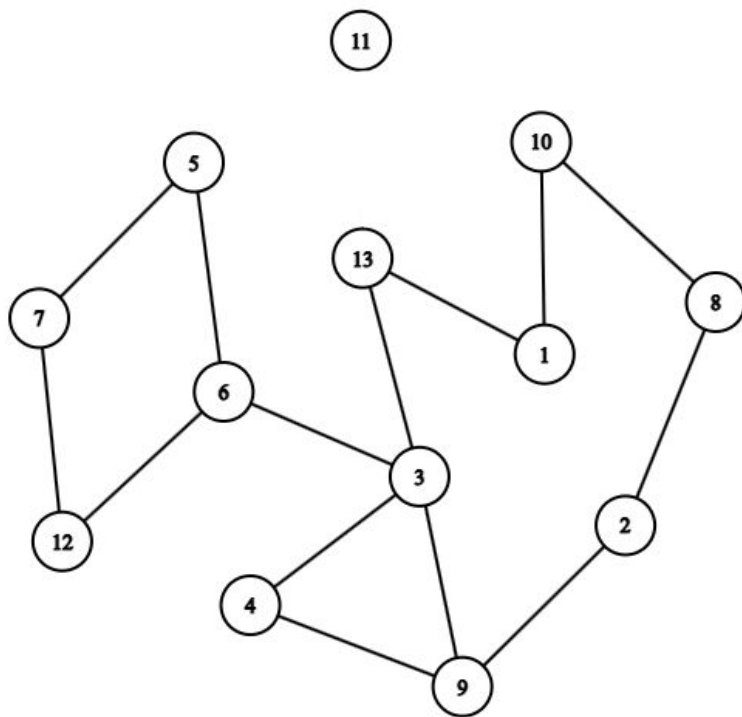
A **bridge** (cut-edge) is an **edge** of the graph whose **deletion** **increases the number** of connected components



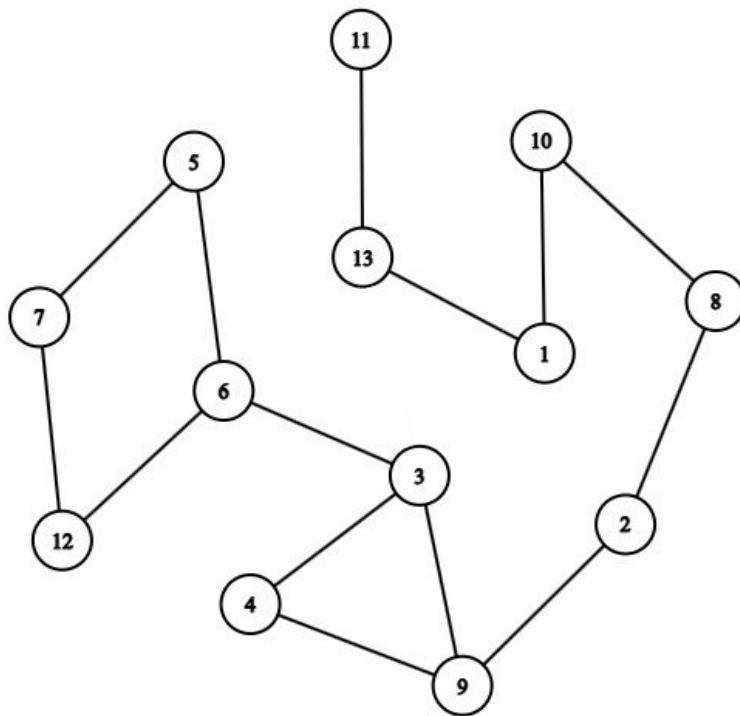
Bridge



Bridge



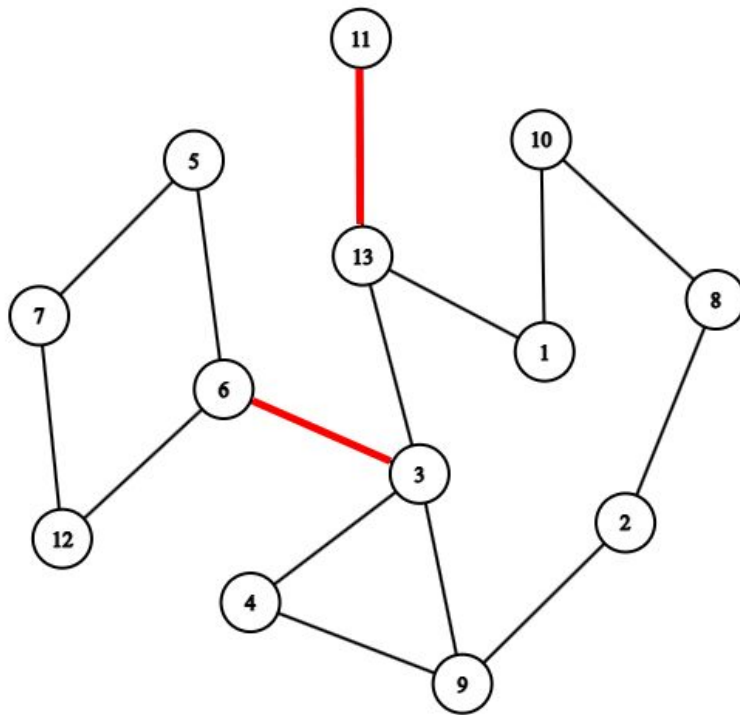
Bridge



Bridge

How many?

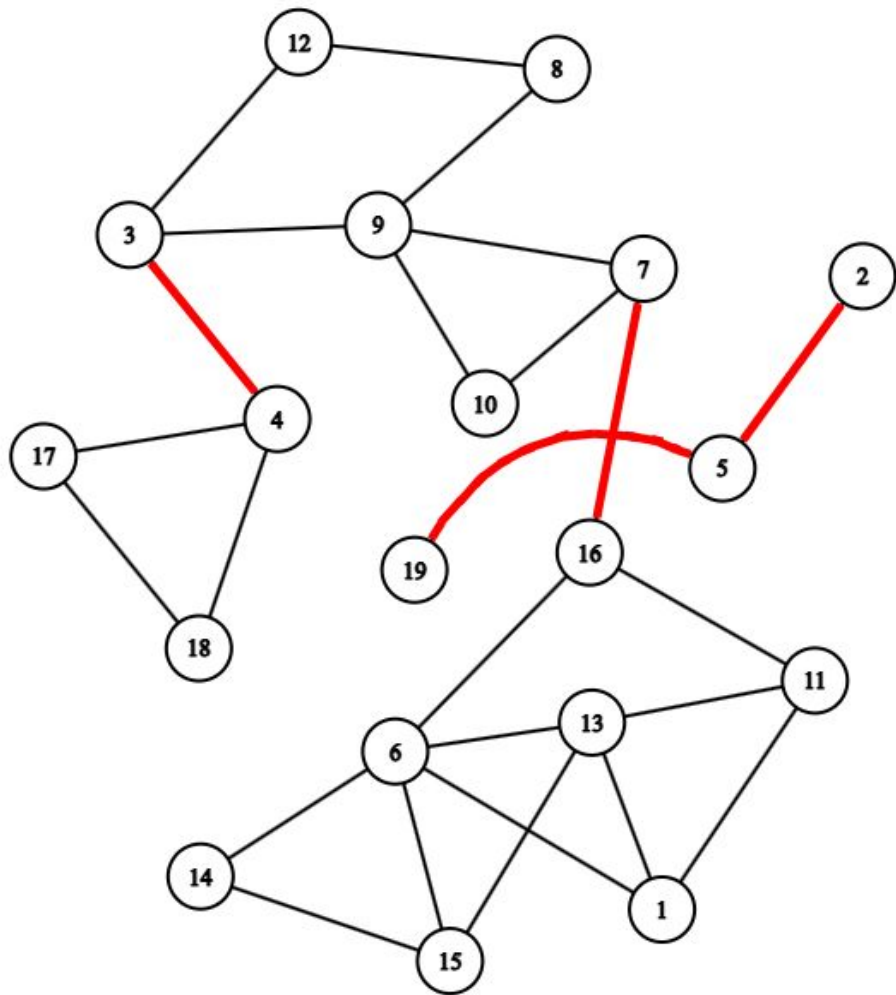
Has 2 bridges!



Bridge

How many bridges?

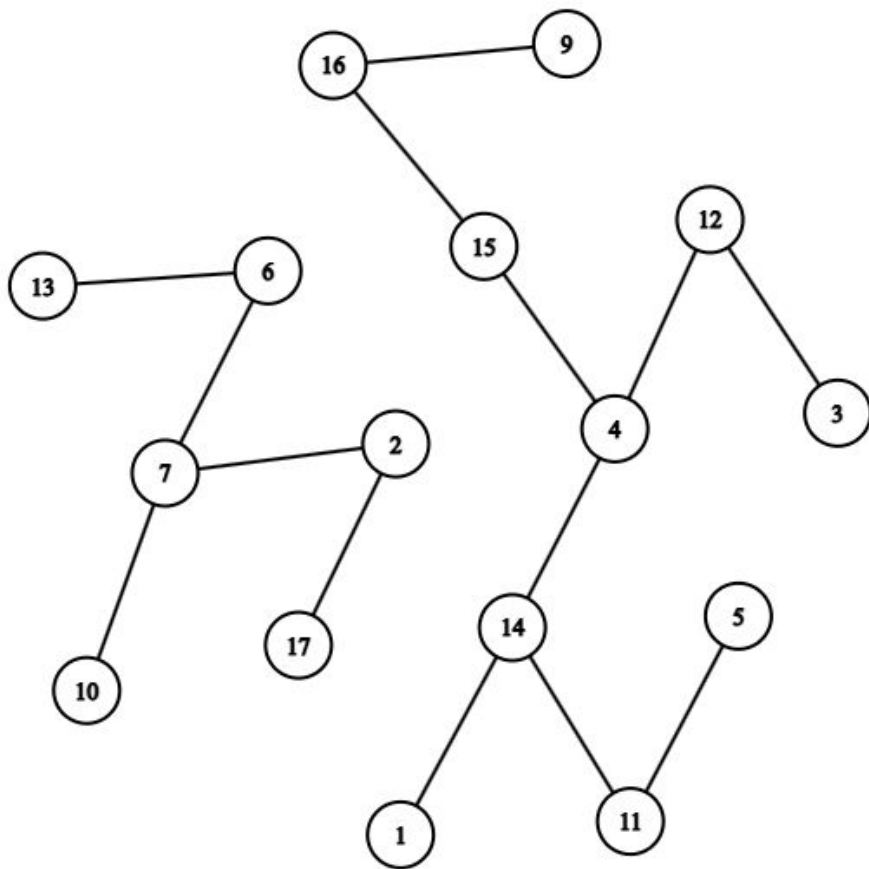
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. 5
- G. 6
- H. 7

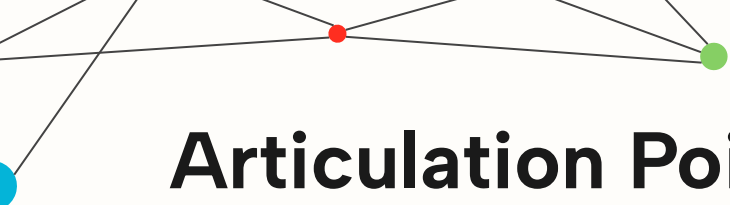


Bridge

In tree (forest), all edges are bridge!

Equivalent definition:
A bridge is an edge that does not belong to any cycle



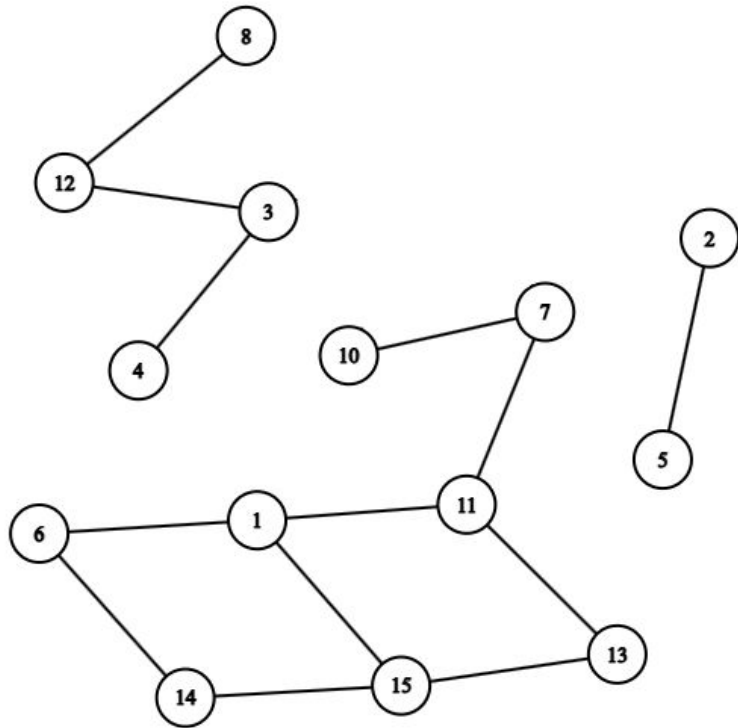


Articulation Point

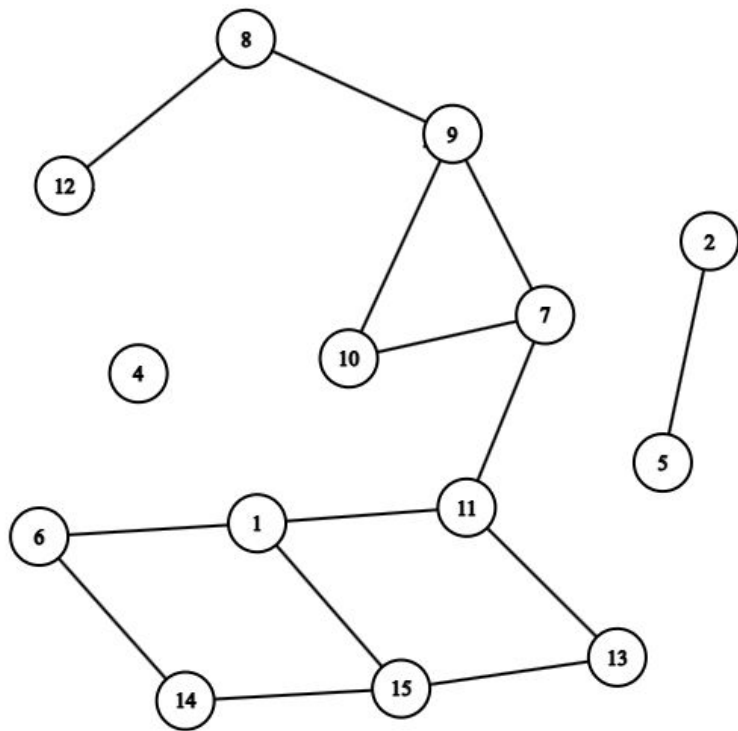
An **articulation point** (cut-vertex, joint) is a **vertex** of the graph whose **deletion increases the number** of connected components



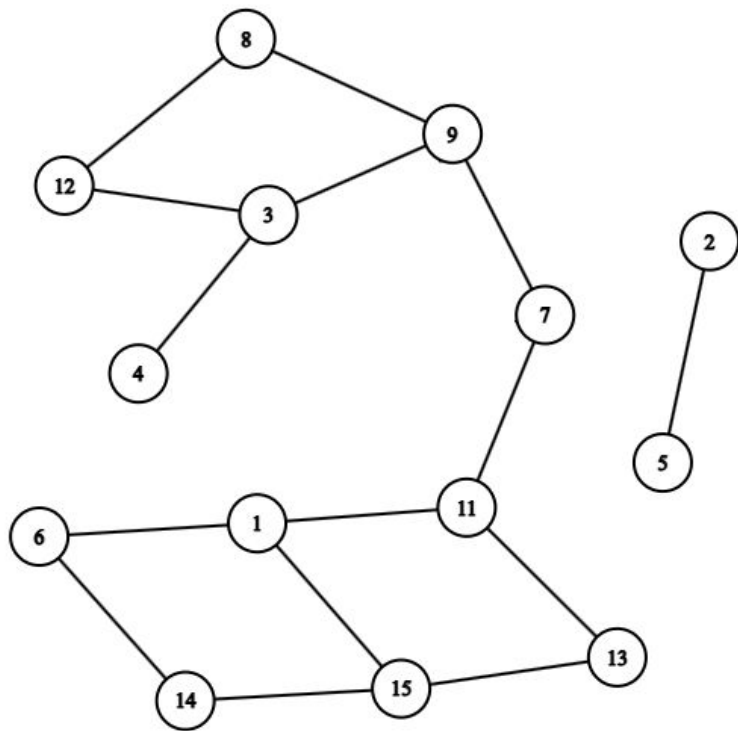
Articulation Point



Articulation Point



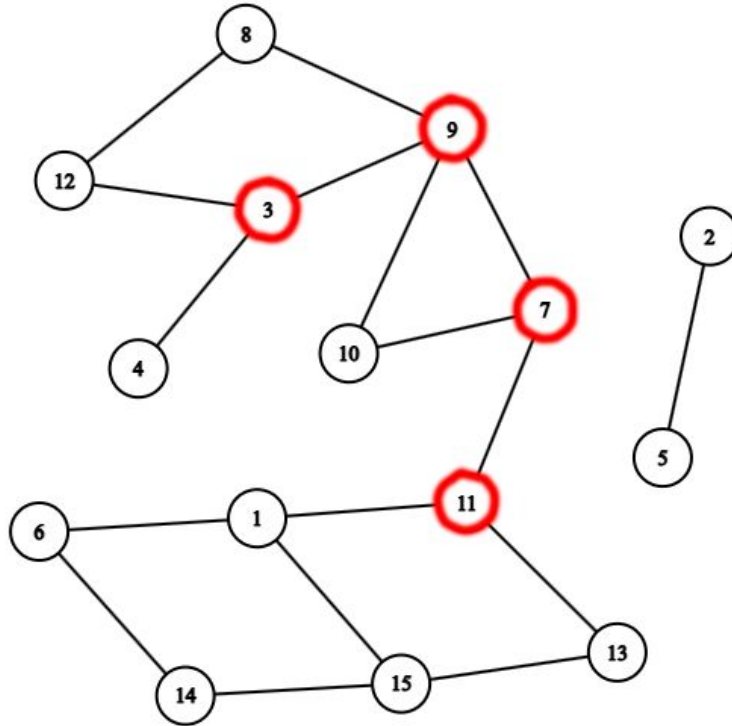
Articulation Point



Articulation Point

How many?

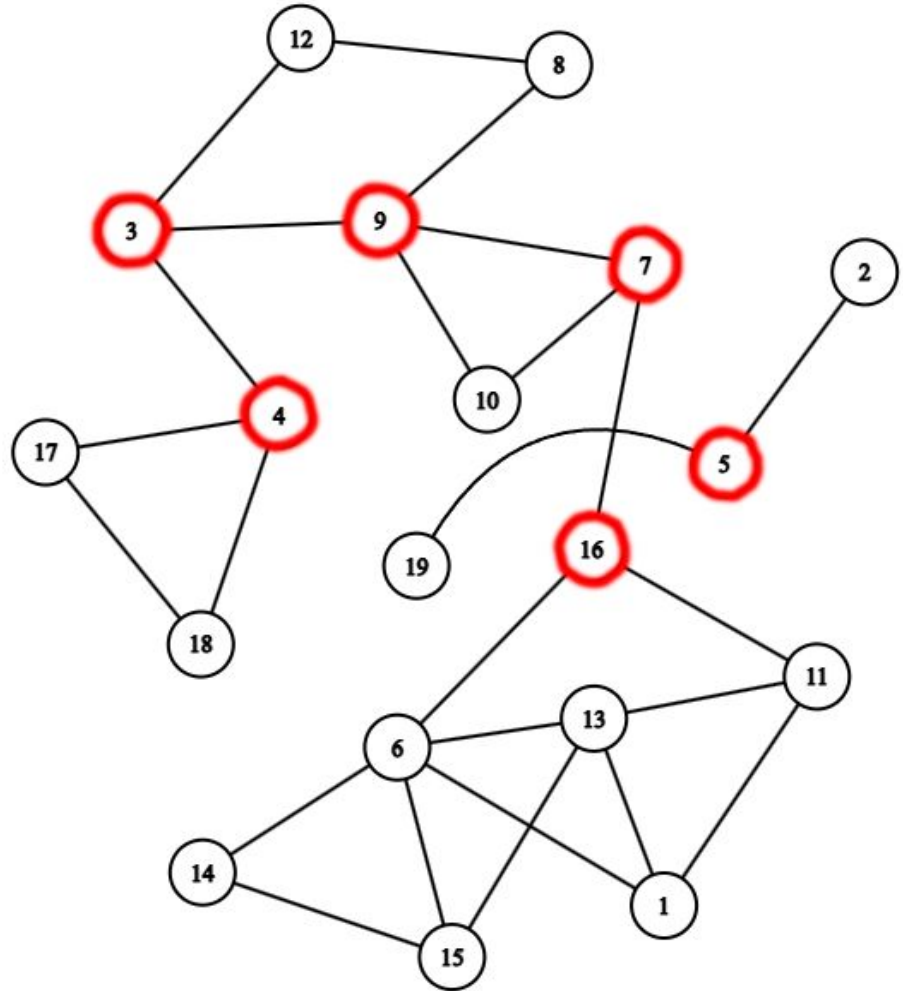
Has 4 articulation points!



Articulation Point

How many articulation points?

- A. 0
- B. 2
- C. 3
- D. 4
- E. 5
- F. 6
- G. 7
- H. 8
- I. 9

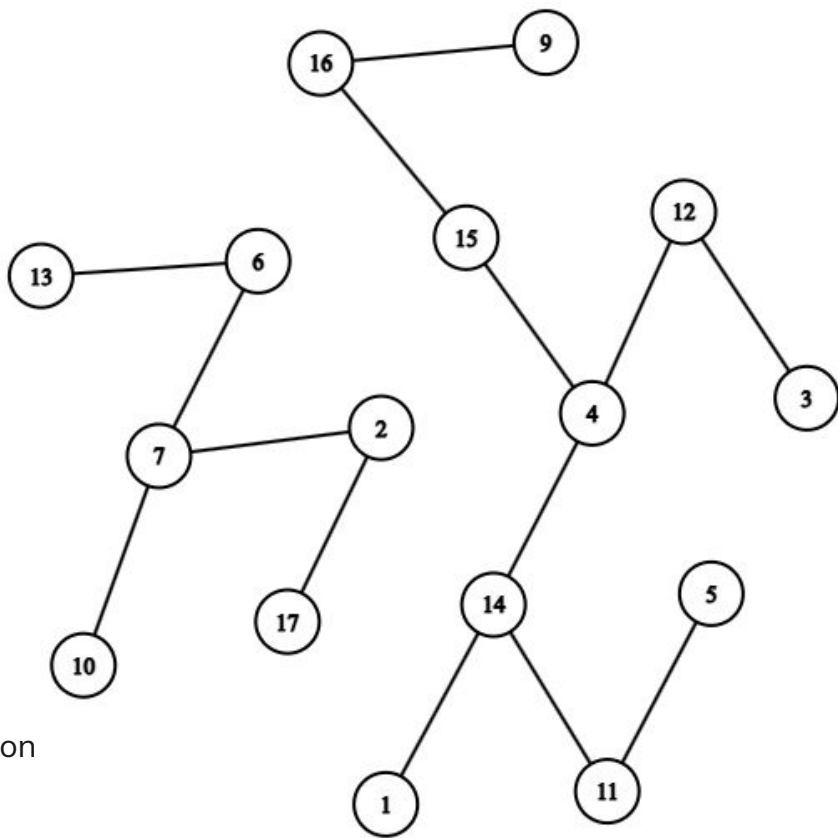


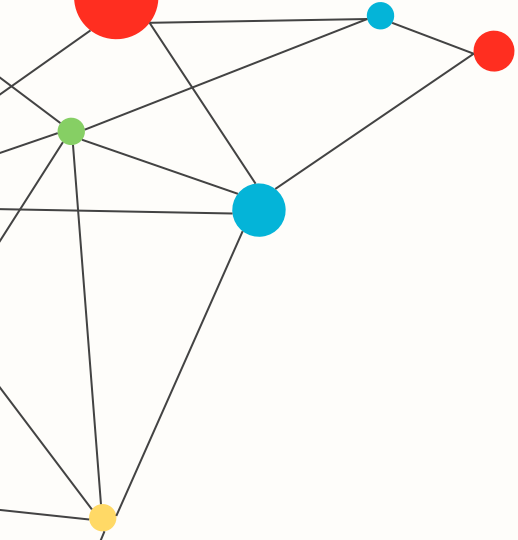
Articulation Point

In a tree (or forest),
any non-leaf nodes is
an articulation point.

Generally, a node
with degree of 1 is
never an articulation
point (in any graph)!

However, a node of degree 0 is never an articulation
point! (removing it decreases #CC, not increase)

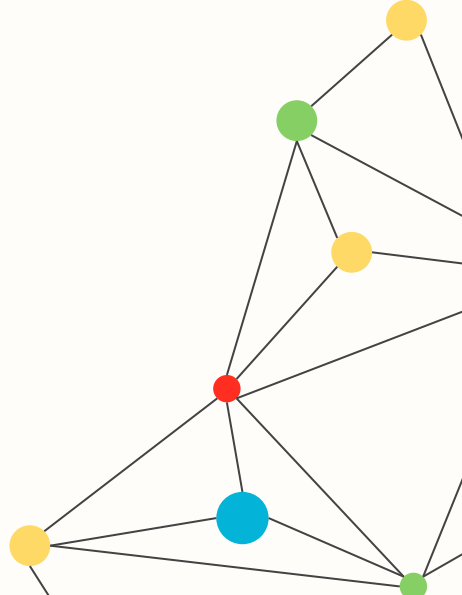




04

Algorithm

The how?





History

During the 1950-1960, people started inspecting network reliability in electrical engineering.

In 1961, L.R. Ford Jr. and D.R. Fulkerson, in their book Flows in Networks, wrote about the understanding of network connectivity and reliability, contributing to future studies.

In 1972, Tarjan presented a linear time algorithms for the computation of bridges and articulation points. [R.E. Tarjan, Depth-first search and linear graph algorithms, SIAM J. Comput. 1 (2) (1972) 146–160.]

Since then, many algorithms are presented that compute structures which inherently characterize the connectivities of a graph.

J. Ebert, st-Ordering the vertices of biconnected graphs, Computing 30 (1983) 19–33.

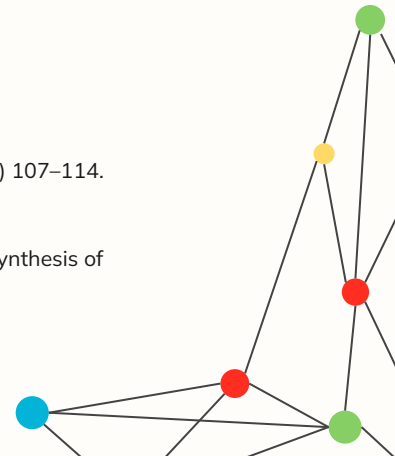
S. Even, R.E. Tarjan, Computing an st-numbering, Theor. Comput. Sci. 2 (3) (1976) 339–344.

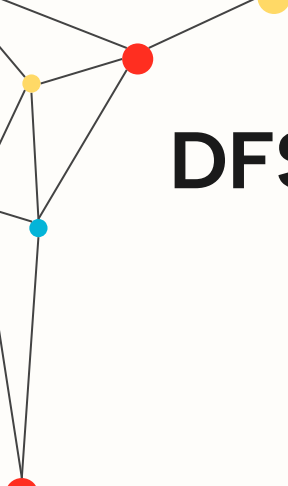
H.N. Gabow, Path-based depth-first search for strong and biconnected components, Inf. Process. Lett. 74 (3–4) (2000) 107–114.

F. Harary, G. Prins, The block-cutpoint-tree of a graph, Publ. Math. Debrecen 13 (1966) 103–107.

Y.H. Tsin, On finding an ear decomposition of an undirected graph distributively, Inf. Process. Lett. 91 (2004) 147–153.

V. Ramachandran, Parallel open ear decomposition with applications to graph biconnectivity and triconnectivity, in: Synthesis of Parallel Algorithms, 1993, pp. 275–340.



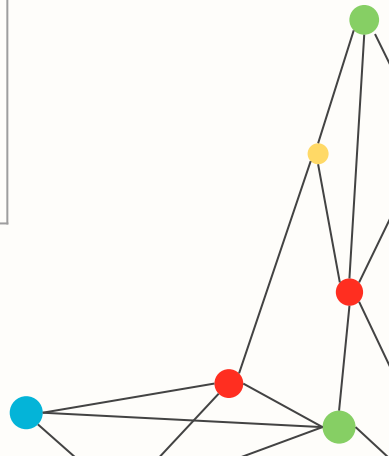


DFS Tree

DFS Tree: Very important to know!

Most algorithms to computing the bridges or articulation points will use this.

It is also used to come up with many other algorithms not related to bridge/articulation point.

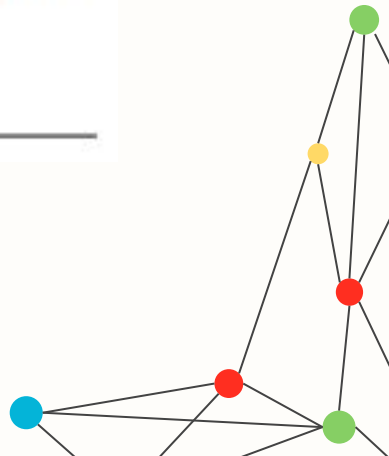




DFS

Algorithm DFS

```
1: function DFS( $G, u$ )  
2:    $visited[u] \leftarrow \mathbf{true}$  ▷ Initially all nodes are unvisited  
3:   for  $v \in G.adjacent(u)$  do  
4:     if  $visited[v] = \mathbf{false}$  then  
5:       DFS( $v$ ) ▷ Recursively visit unvisited node  
6:     end if  
7:   end for  
8: end function
```



DFS Tree

Here, make (u, v) a tree-edge

Algorithm DFS

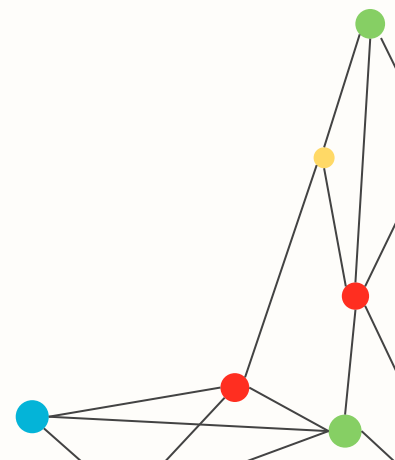
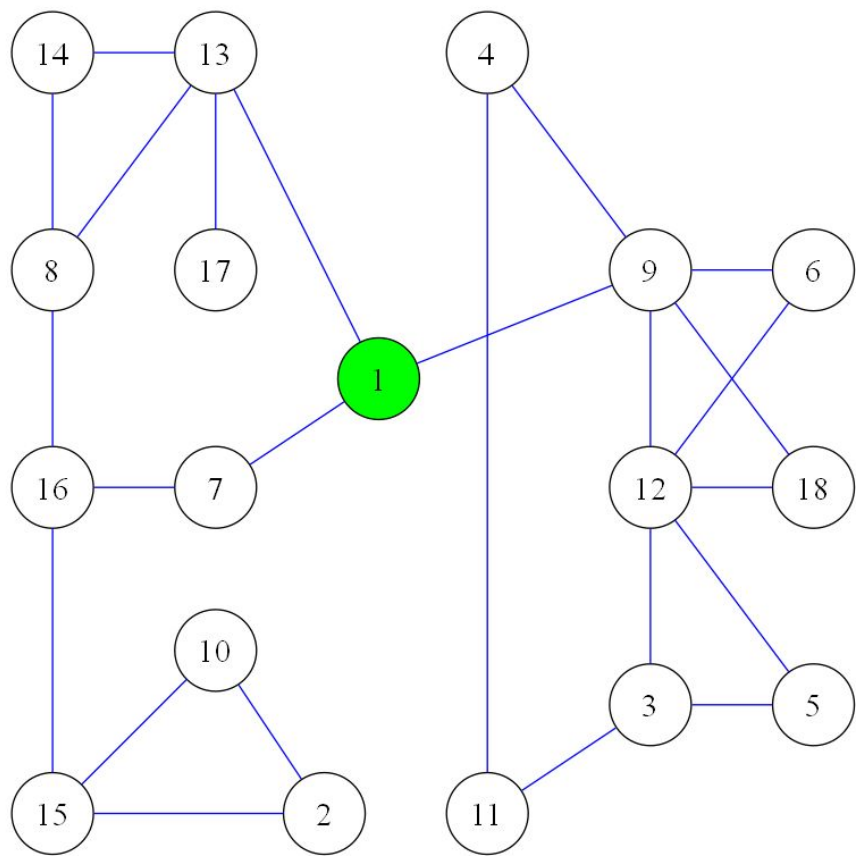
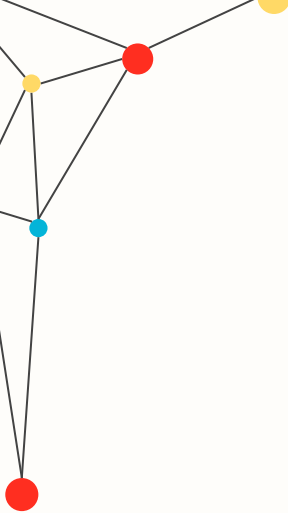
```
1: function DFS( $G, u$ )  
2:    $visited[u] \leftarrow \mathbf{true}$   
3:   for  $v \in G.adjacent(u)$  do  
4:     if  $visited[v] = \mathbf{false}$  then  
5:       DFS( $v$ )  
6:     end if  
7:   end for  
8: end function
```

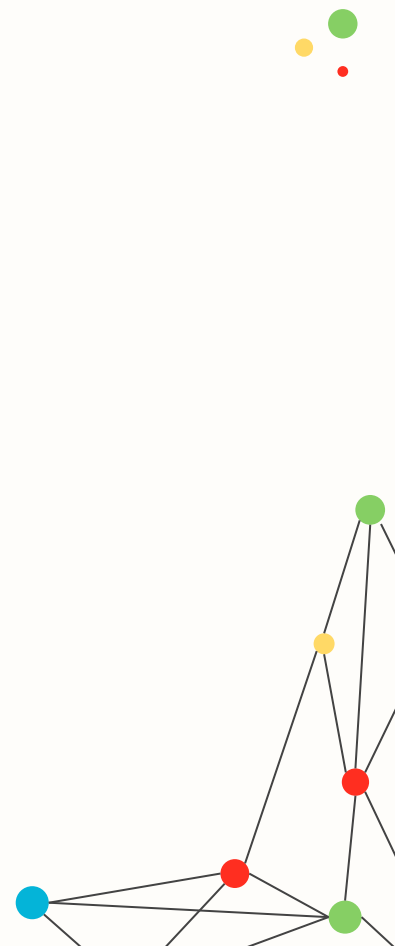
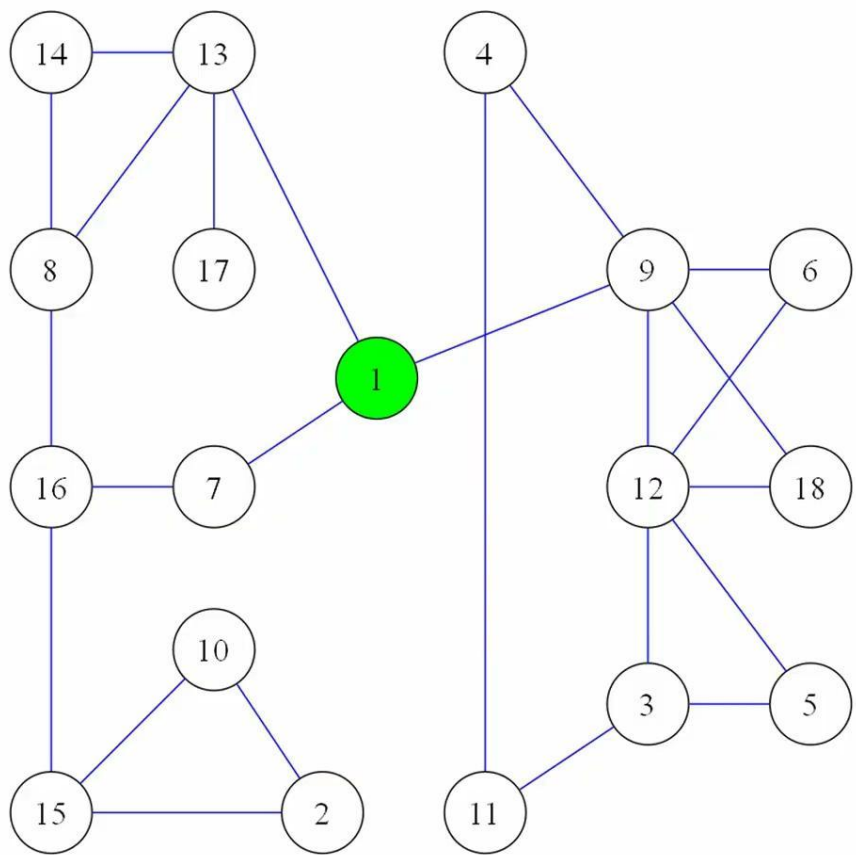
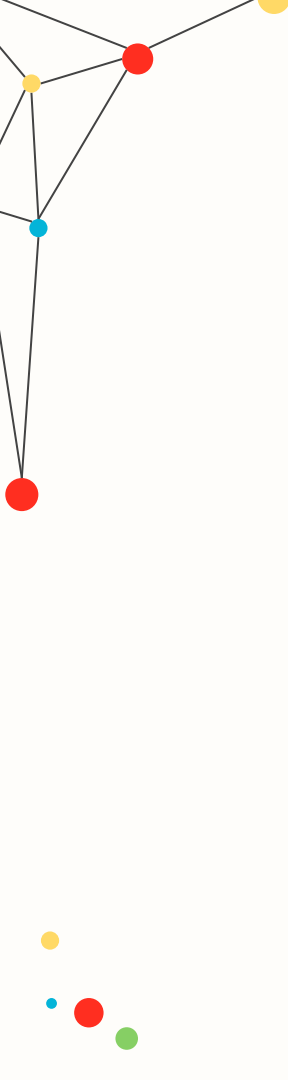
Non tree-edges will be called **cotree-edge**

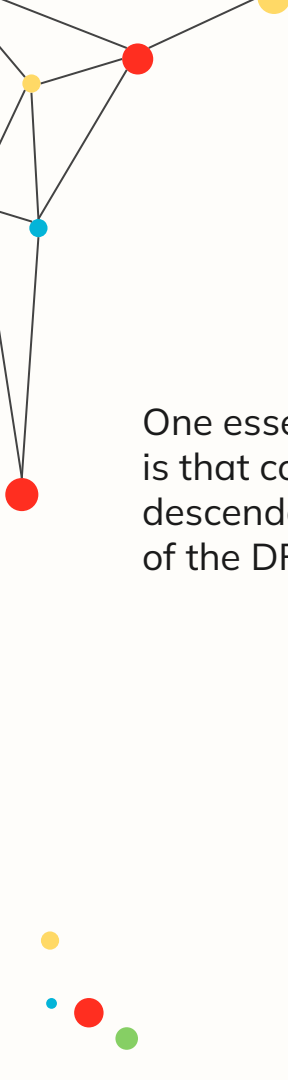
DFS Tree

Algorithm DFS Tree Construction

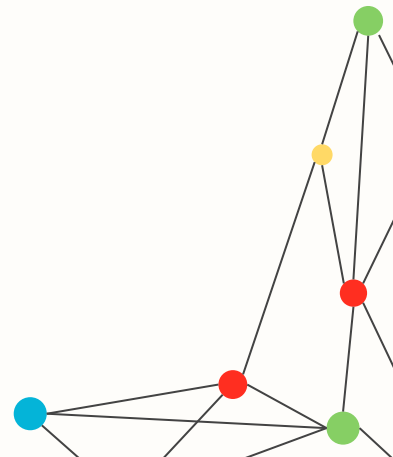
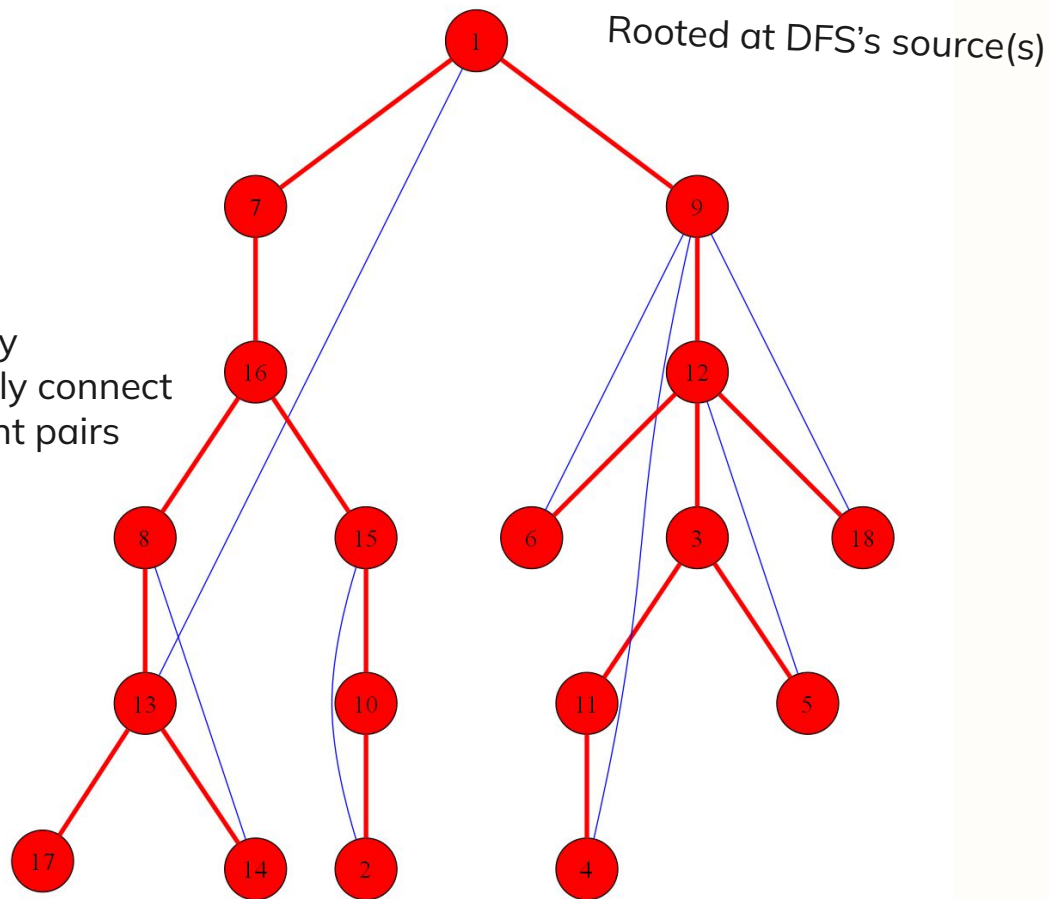
```
1: function DFS_TREE( $G, root$ )
2:    $visited \leftarrow$  array of  $|G.V|$  booleans initialized to false
3:    $tree\_edges \leftarrow$  empty set
4:    $cotree\_edges \leftarrow$  empty set
5:
6:   function DFS( $u$ )
7:      $visited[u] \leftarrow$  true
8:     for each  $v \in G.adjacent(u)$  do
9:       if  $visited[v] = \text{false}$  then
10:        add  $(u, v)$  to  $tree\_edges$ 
11:        DFS( $v$ )
12:       else if  $(u, v) \notin tree\_edges$  then
13:        add  $(u, v)$  to  $cotree\_edges$ 
14:       end if
15:     end for
16:   end function
17:
18:   DFS( $root$ )
19:   return ( $tree\_edges, cotree\_edges$ )
20: end function
```





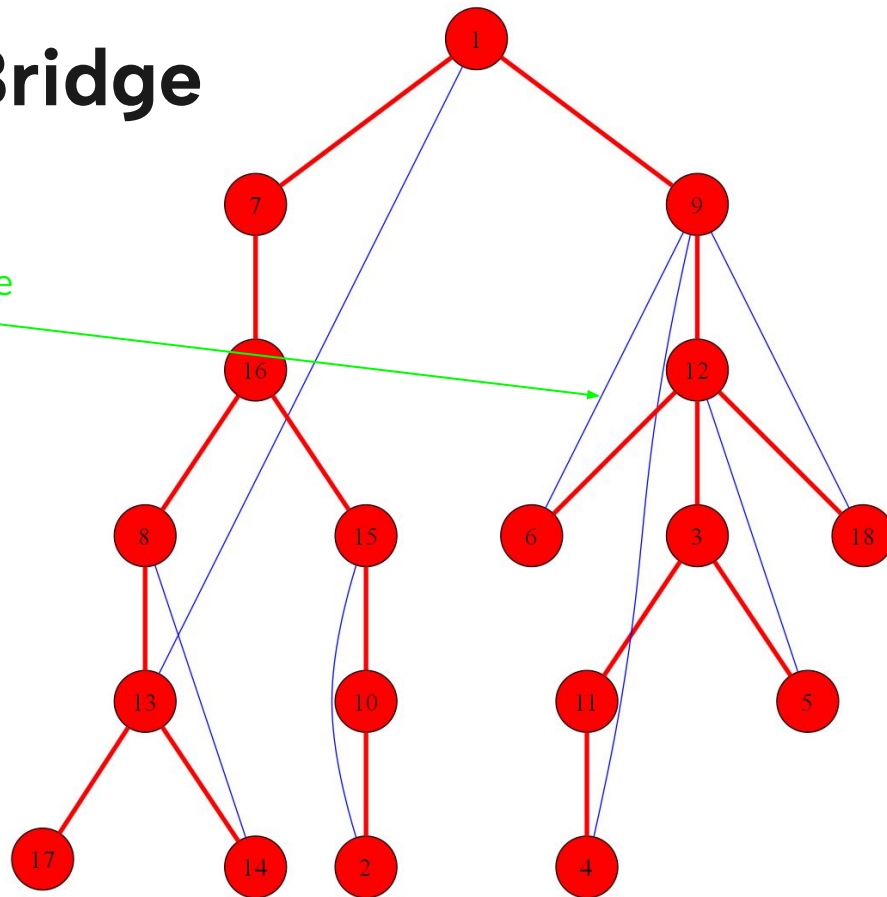


One essential property is that cotree edge only connect descendant-ancestor pairs of the DFS tree!



Finding Bridge

Cotree edges are
not bridge!

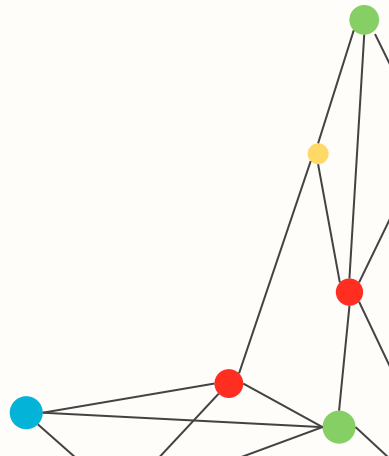




Finding Bridge

1. Cotree edges are not bridges.

2. Tree edges are bridges iff there is no cotree edge
“passing over” it



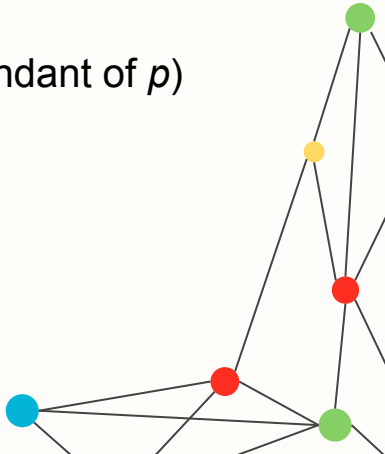


Finding Bridge

1. Cotree edges are not bridges.

2. Tree edges are bridges iff there is no cotree edge “passing over” it

(a tree edge (p, u) is bridge iff no descendant of u connects to an ascendant of p)

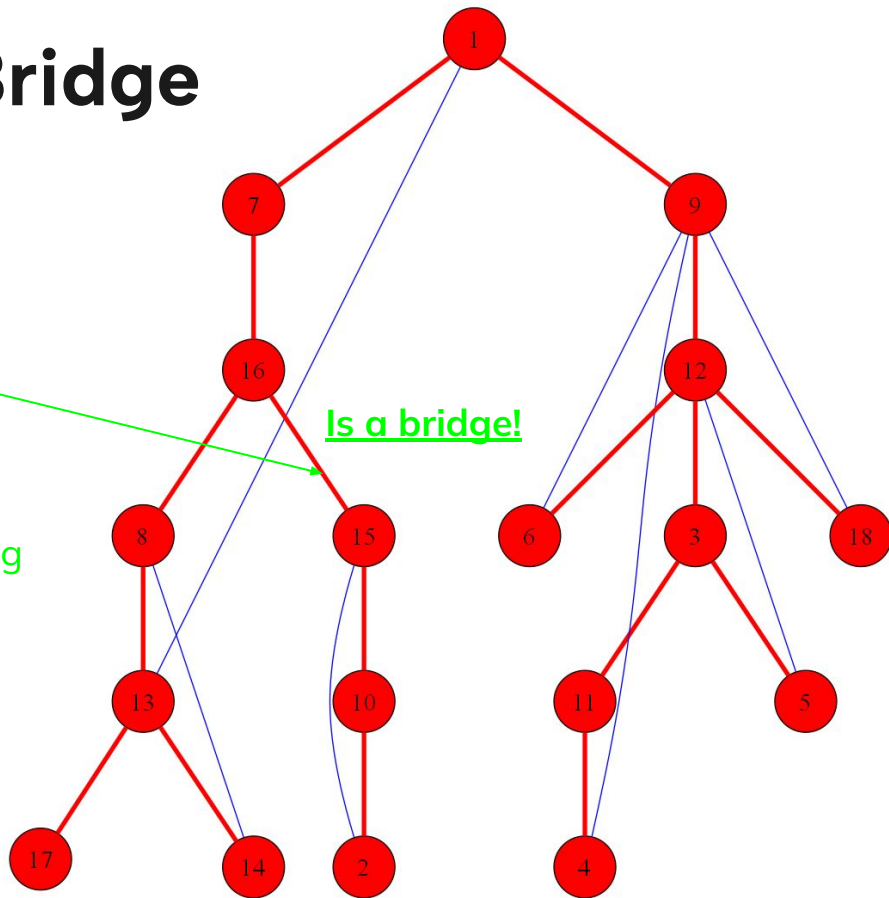


Finding Bridge

Consider this edge

No cotree edge passing over.

Is a bridge!

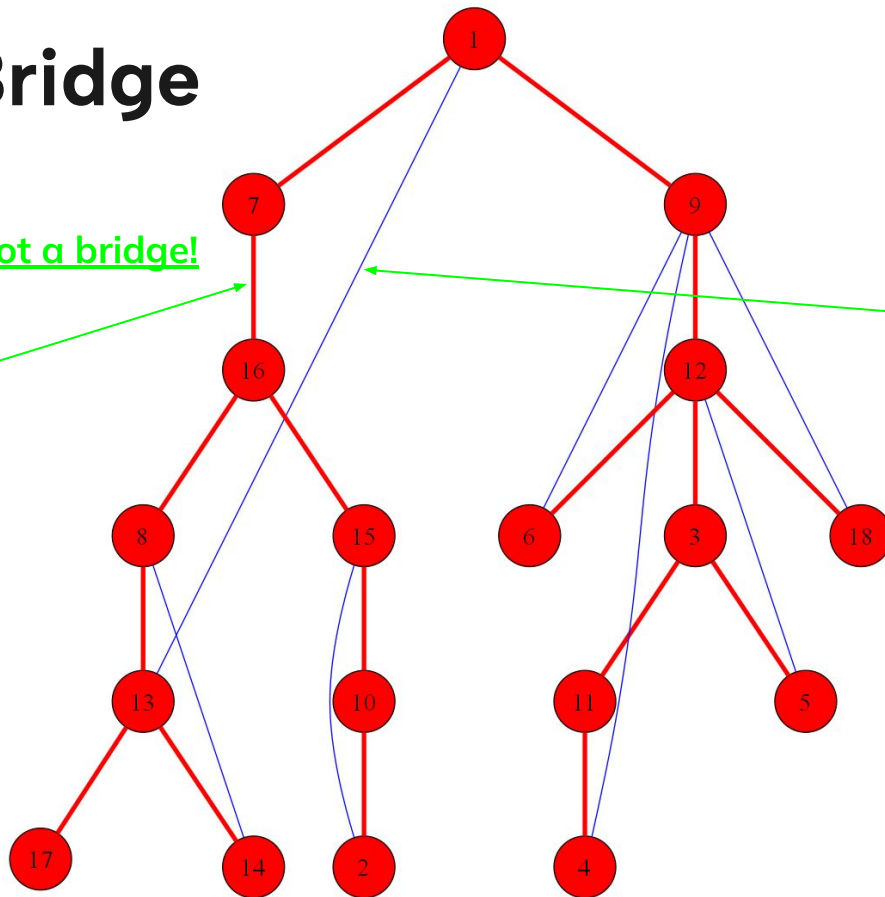


Finding Bridge

Not a bridge!

Consider this edge

Cotree edge
"passing over"!





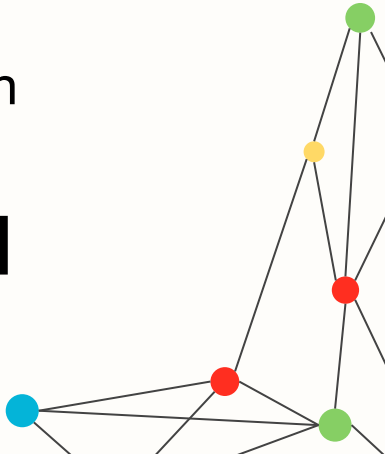
Finding Bridge

How do we know the “passing over”?

Let $\text{reach}[u] :=$ the lowest depth (nearest to root) that some node in the subtree of u can go to by zero or one cotree edge.
(and no tree-edge)

If $\text{reach}[u] \geq \text{depth}[u]$ then we know no cotree edge in subtree of u goes outside. (with $p = \text{parent}[u]$)

→ **(p, u) is a bridge iff $\text{reach}[u] \geq \text{depth}[u]$**





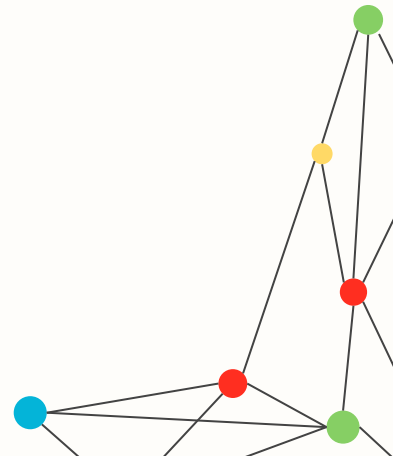
Finding Bridge

To compute $\text{reach}[]$

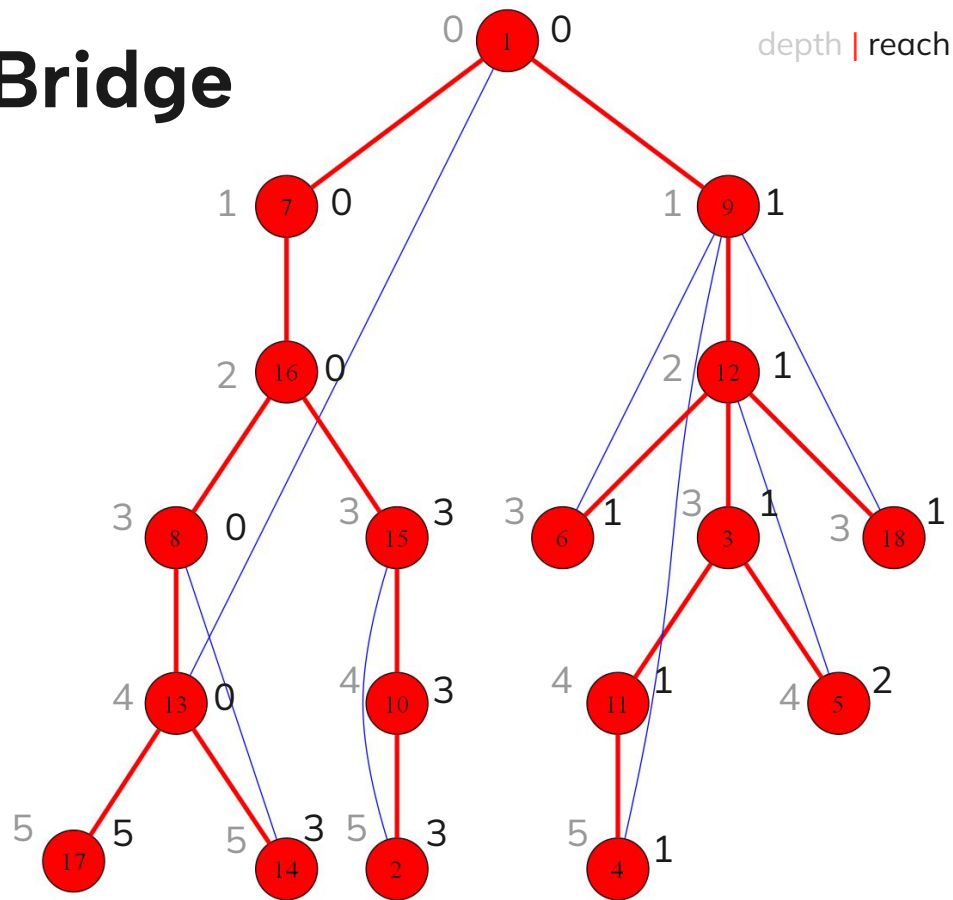
Recall: $\text{reach}[u] :=$ the lowest depth (nearest to root) that some node in the subtree of u can go to by zero or one cotree edge.
(and no tree-dge)

$\text{reach}[u] =$ minimum of the followings:

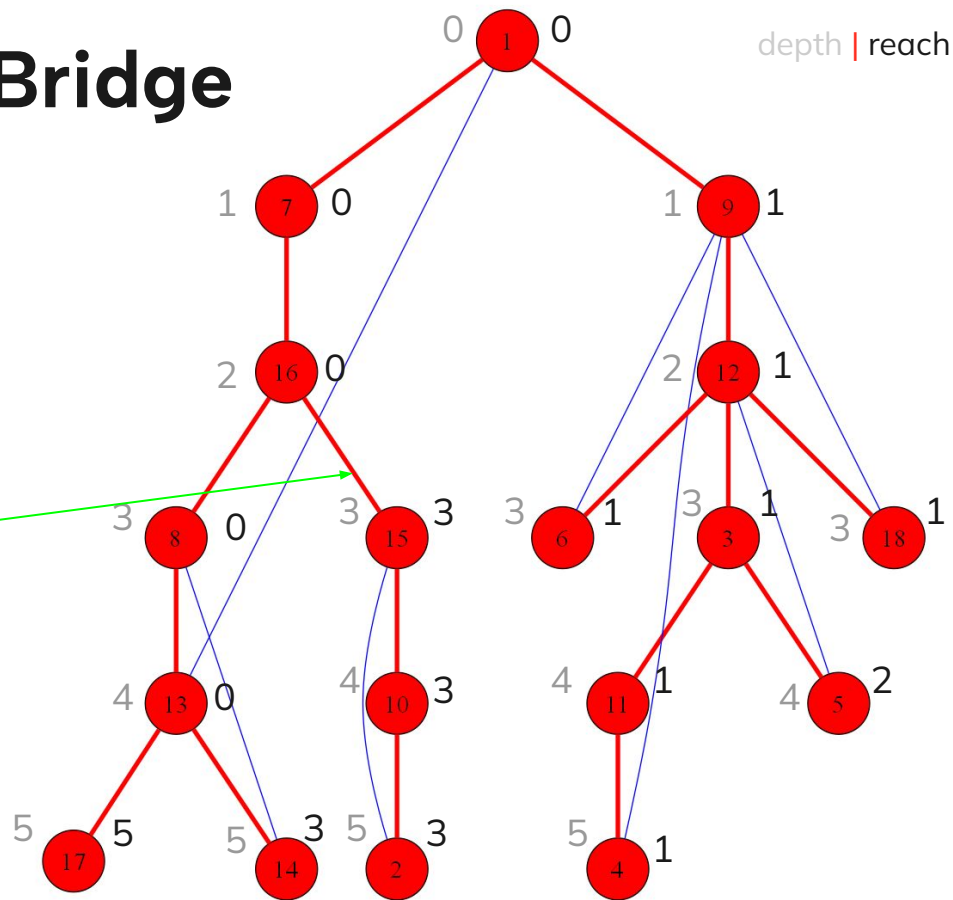
- + $\text{depth}[u]$
- + $\text{depth}[w]$ for some $(u, w) \in \text{cotree_edges}$
- + $\text{reach}[v]$ for some child v of u



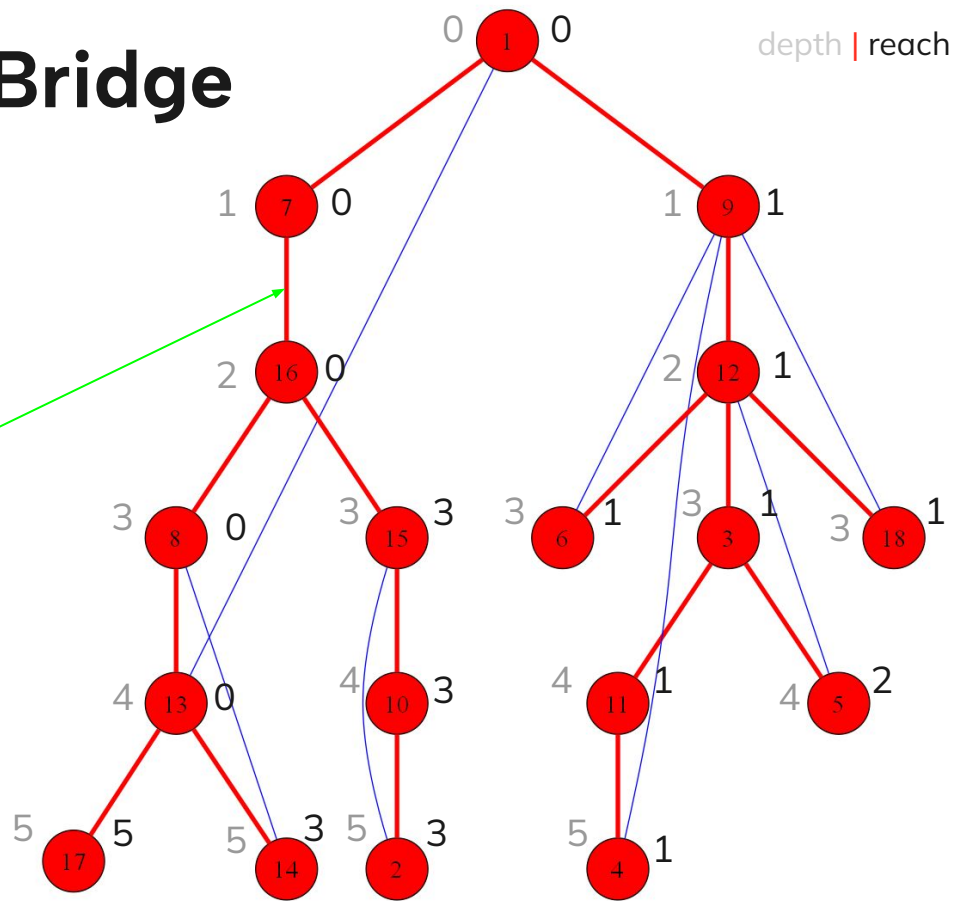
Finding Bridge



Finding Bridge



Finding Bridge



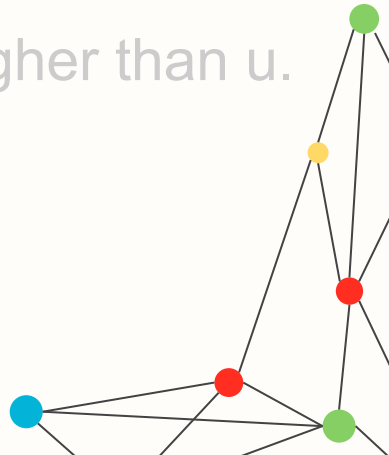


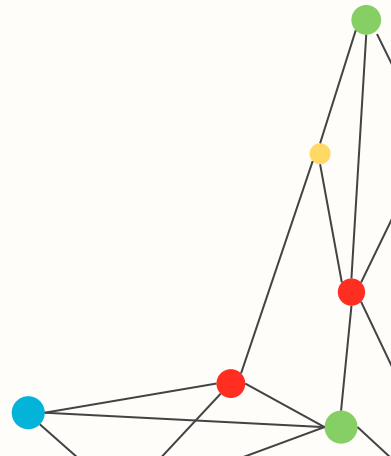
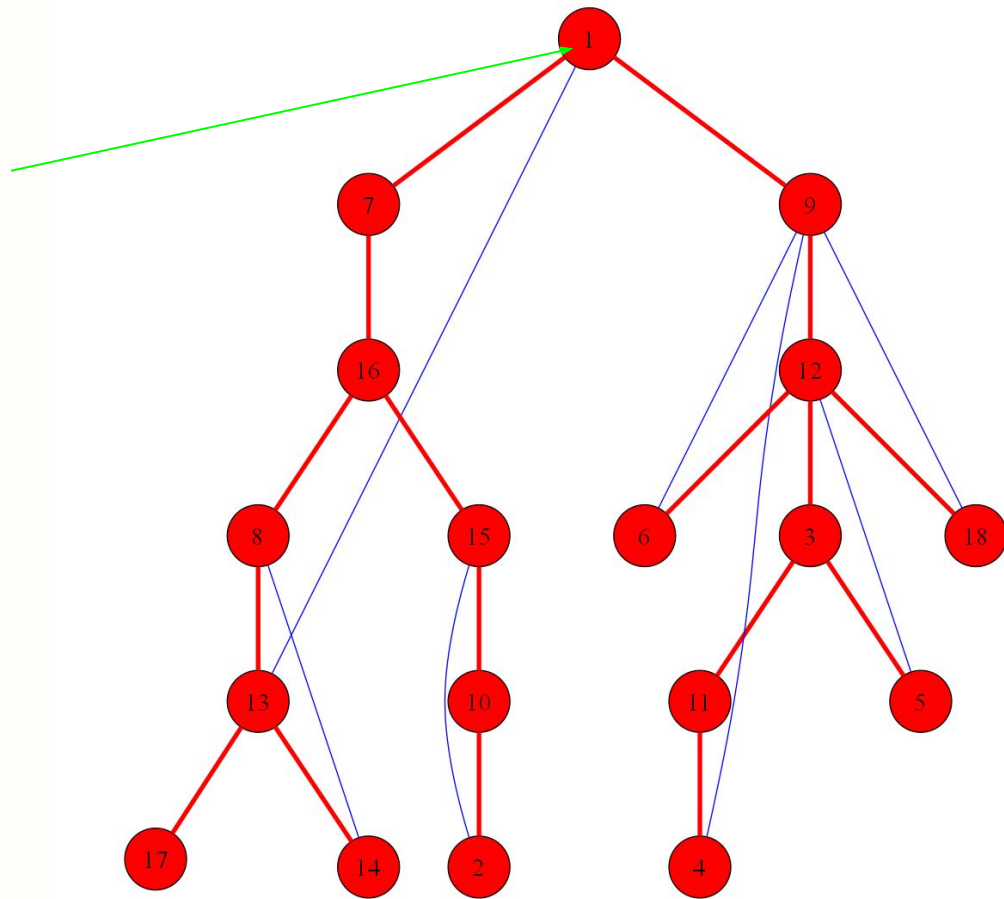
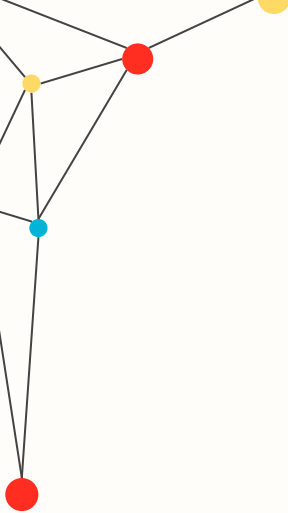
Finding Articulation Point

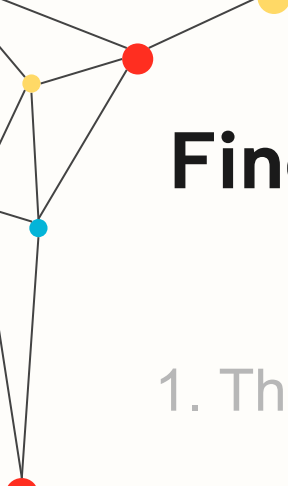
1. The root is an articulation iff it has at least 2 children.

(recall that nodes with degree ≤ 1 are not articulation points)

2. Non-root u is an articulation point iff there is a child v whose subtree does not have a cotree edge connecting to higher than u .





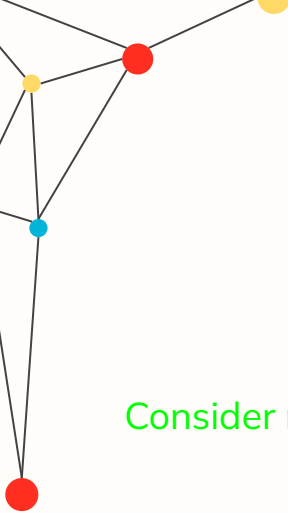


Finding Articulation Point

1. The root is an articulation iff it has at least 2 children.

2. Non-root u is an articulation point iff there is a child v whose subtree does not have a cotree edge connecting to higher than u . (to some proper ascendant of u)

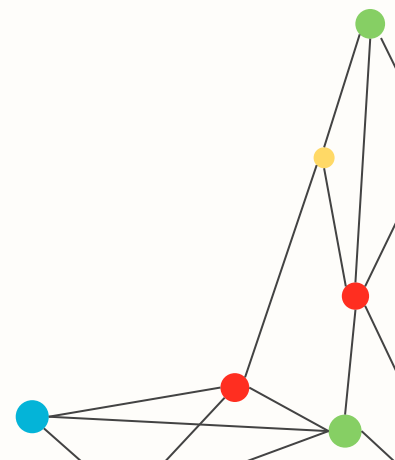
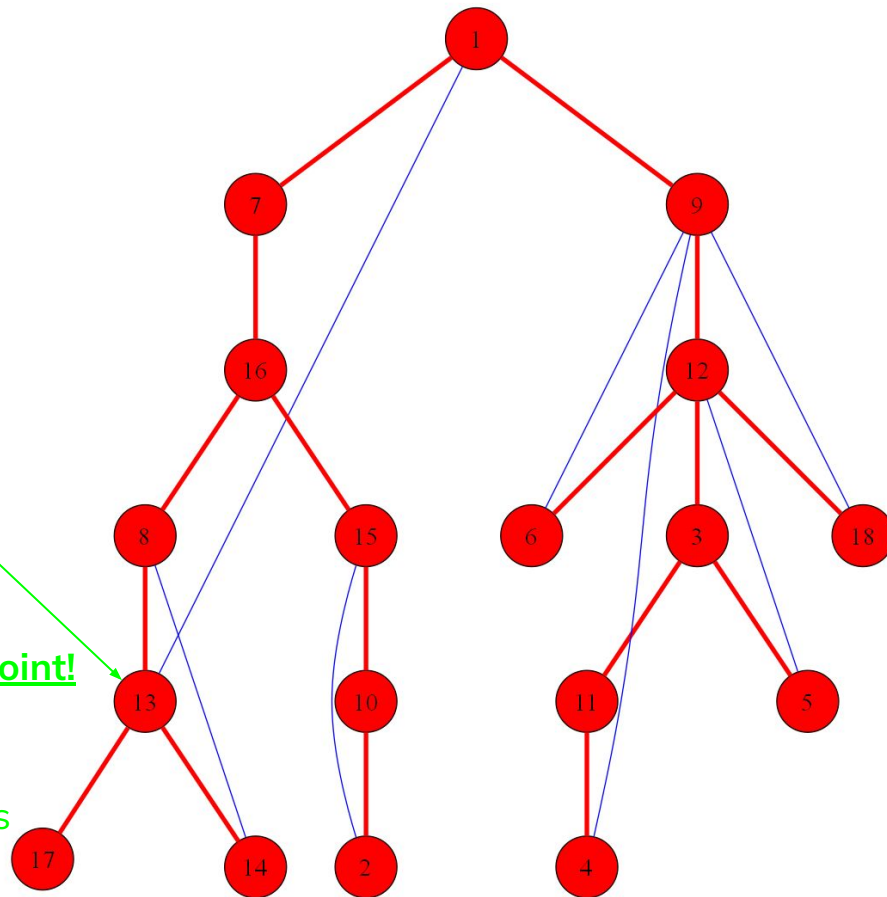




Consider node 13

Is an articulation point!

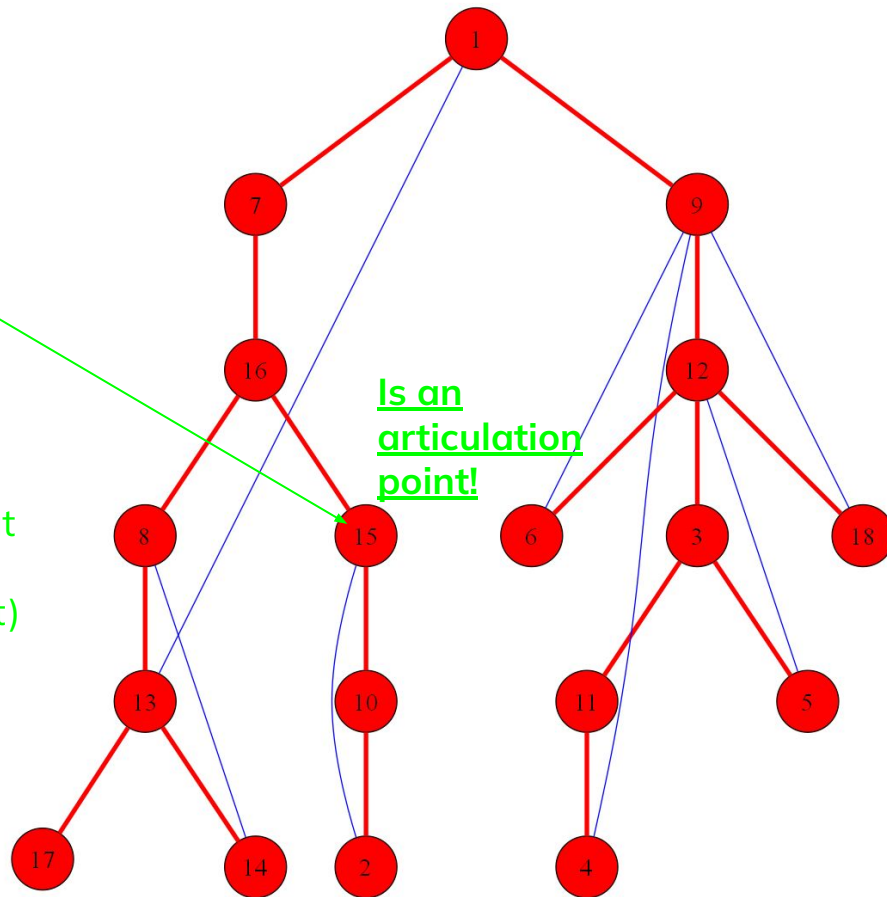
Child 17 does not goes
out of subtree 13

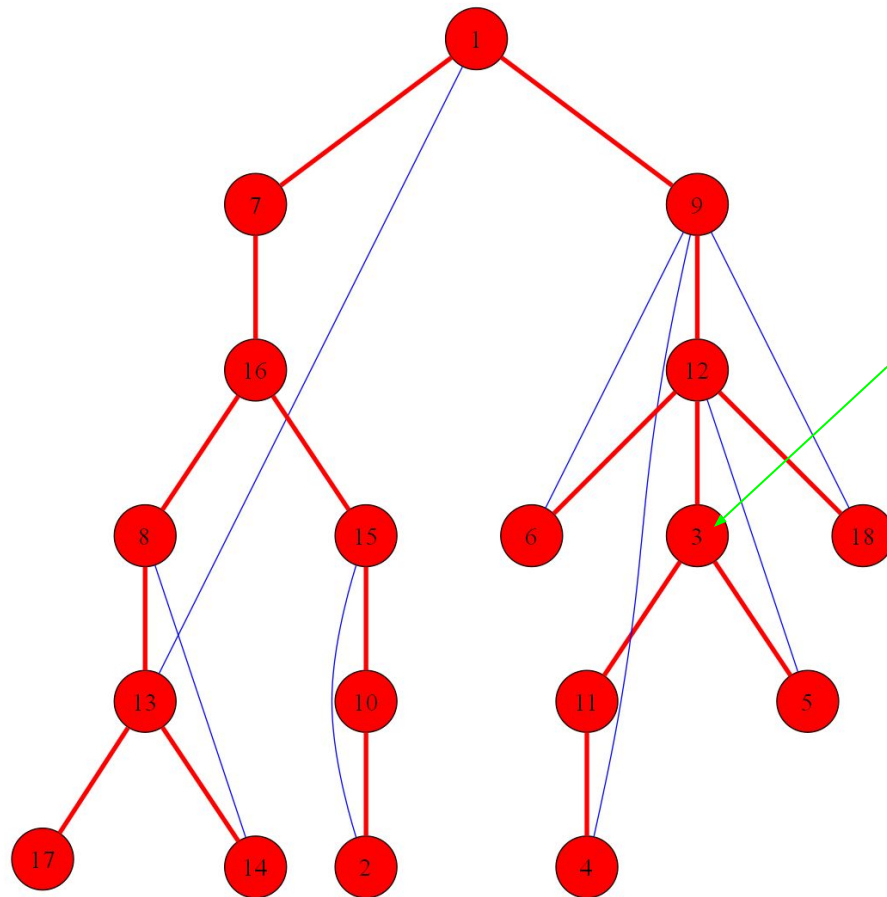


Consider node 15

Its child does not goes out
of subtree 15
(only up to 15, not out yet)

Is an
articulation
point!





Consider node 3

Both its children does
goes out of subtree 3

3 Is NOT an
articulation
point!



Finding Articulation Point

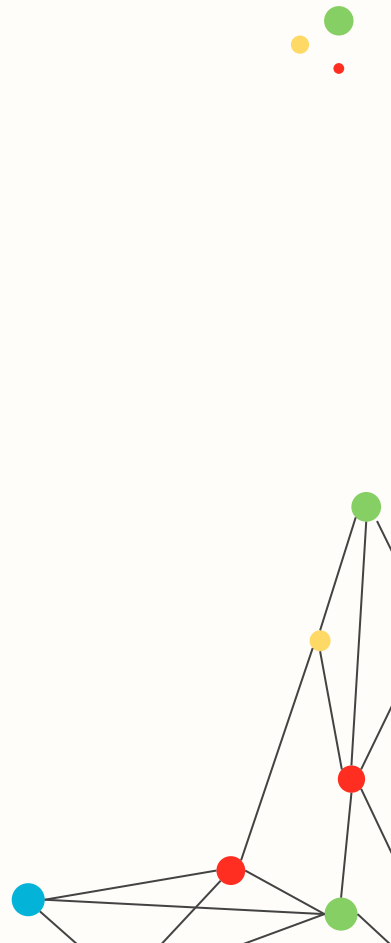
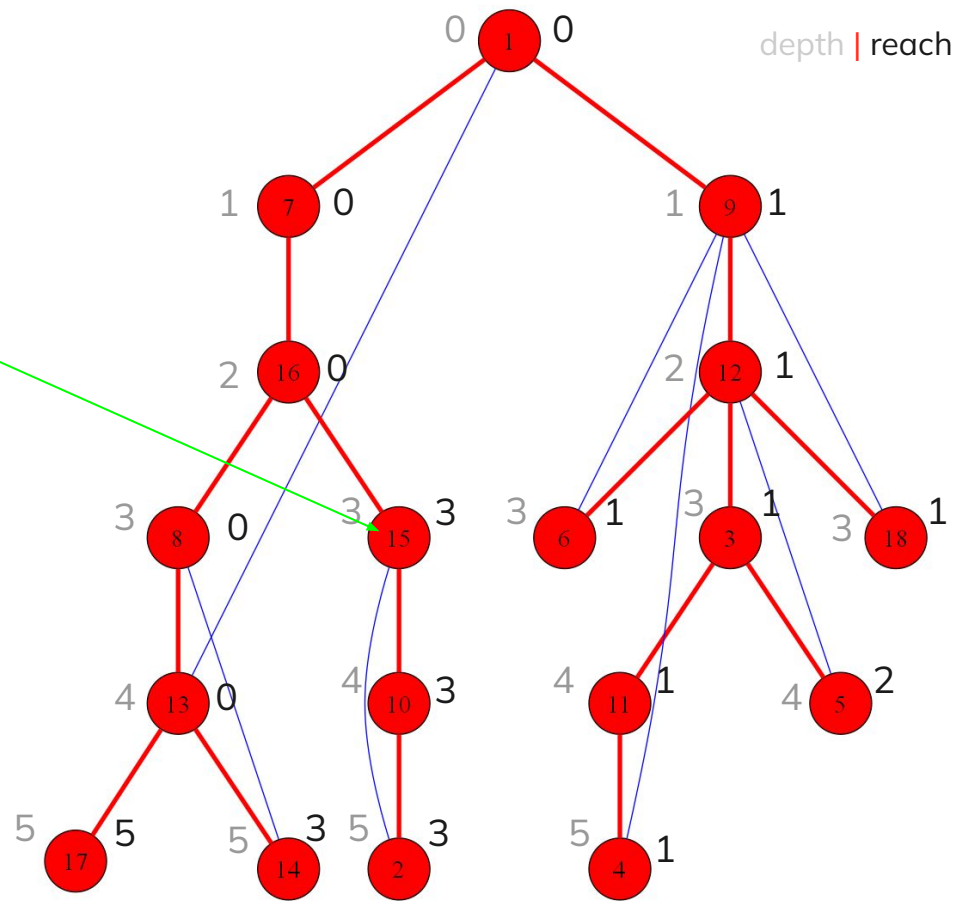
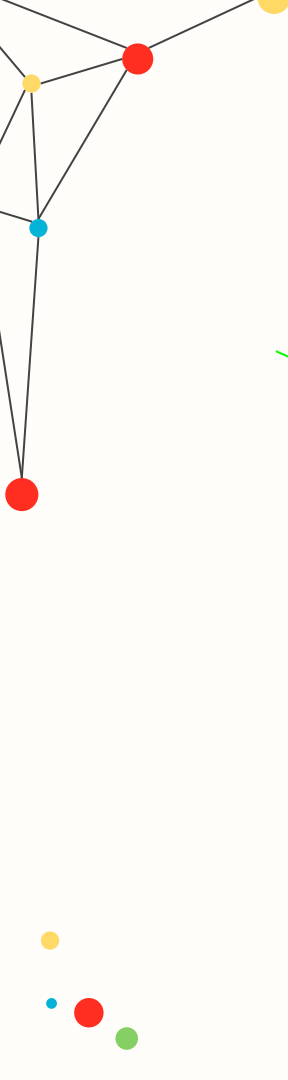
We will reuse the `reach[]`

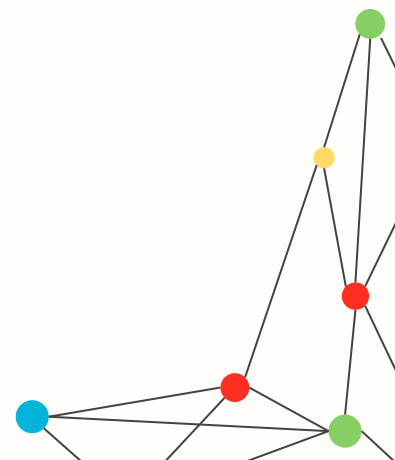
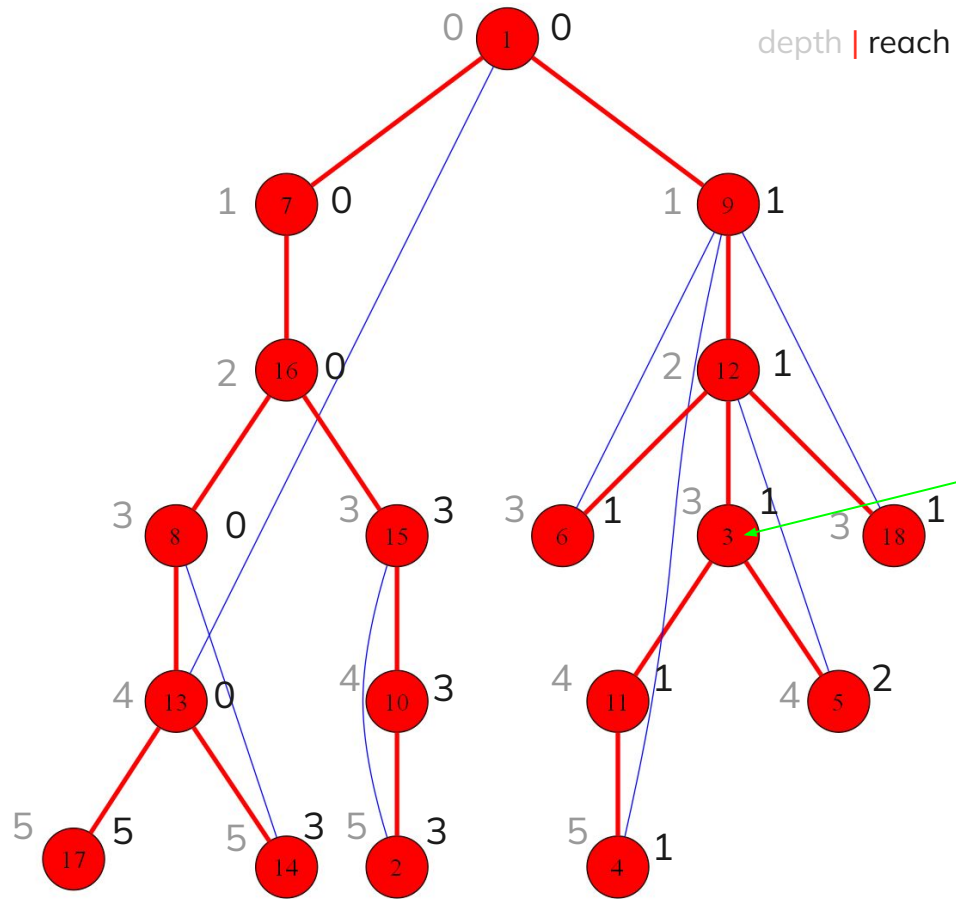
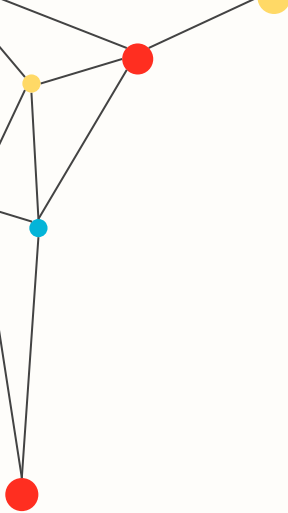
(`reach[u]` := the lowest depth (nearest to root) that some node in the subtree of `u` can go to by zero or one cotree edge)

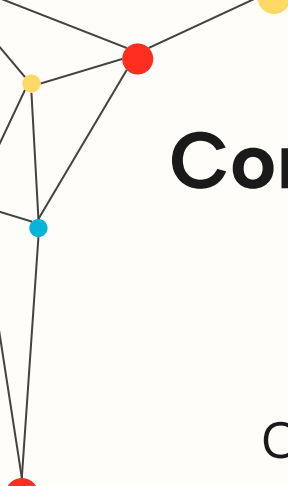
For a node `u` and its child `v`, if $\text{reach}[v] \geq \text{depth}[u]$ then we know no cotree edge in subtree of `v` goes outside of `u`.

→ **`u` is an articulation point iff there exist at least one child `v` where $\text{reach}[v] \geq \text{depth}[u]$**









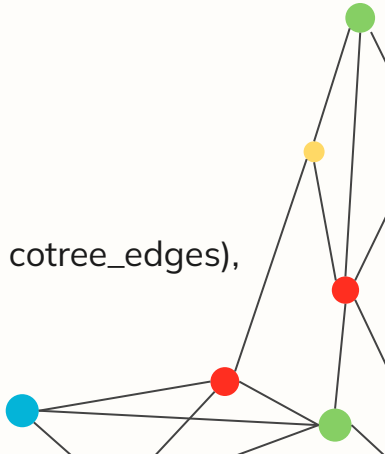
Complexity?

Constructing the DFS Tree and computing its' properties (depth and reach) is only one single pass of DFS each.

Time complexity: $O(|V| + |E|)$

Memory complexity: $O(|V| + |E|)$ ^(*)

^(*) for the implementation that doesn't explicitly compute the DFS tree (tree_edges, cotree_edges), the additional memory is only $O(|V|)$ for depth[] and reach[]



DFS Tree

Some note on the implementation

Algorithm DFS Tree Construction

```
1: function DFS_TREE( $G, root$ )
2:    $visited \leftarrow$  array of  $|G.V|$  booleans initialized to false
3:    $tree\_edges \leftarrow$  empty set
4:    $cotree\_edges \leftarrow$  empty set
5:
6:   function DFS( $u$ )
7:      $visited[u] \leftarrow$  true
8:     for each  $v \in G.adjacent(u)$  do
9:       if  $visited[v] = \text{false}$  then
10:        add  $(u, v)$  to  $tree\_edges$ 
11:        DFS( $v$ )
12:       else if  $(u, v) \notin tree\_edges$  then
13:        add  $(u, v)$  to  $cotree\_edges$ 
14:       end if
15:     end for
16:   end function
17:
18:   DFS( $root$ )
19:   return ( $tree\_edges, cotree\_edges$ )
20: end function
```

Should differentiate the edges by their ID to avoid usage of heavy DS (std::set) and use marking array instead

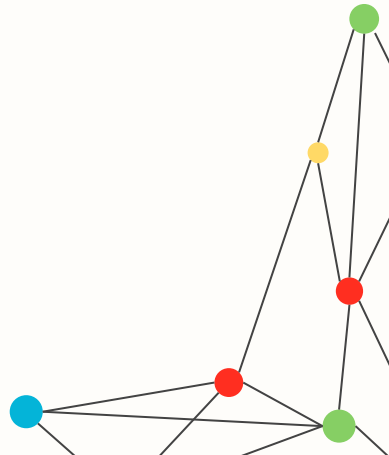
This will also handle “back-tracing” an undirected edge when using adjacency list, as well as properly identifying multi-edge.

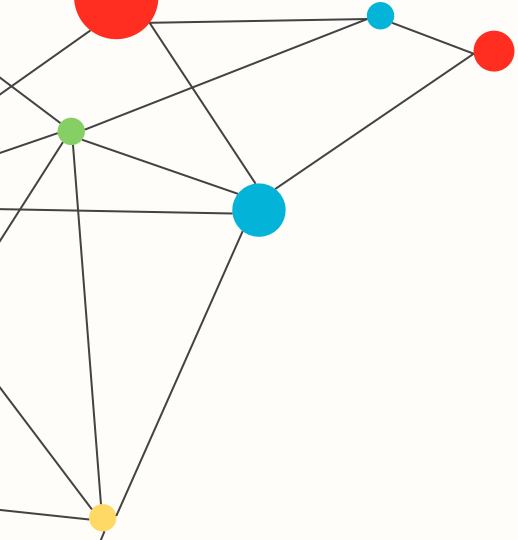
Note: the only cases where (u, v) is in $tree_edges$ is when we are trying to back-trace to the parent (or multi-edges).



Other algorithms?

- + Chain Decomposition
- + Online Bridge Finding (DSU)
- + Any spanning tree
- + ...

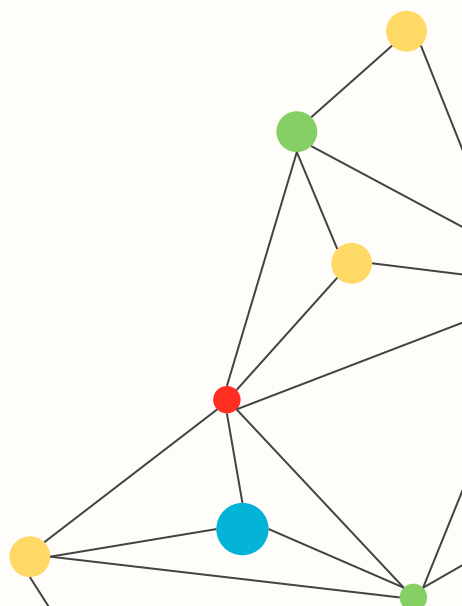




05

Applications

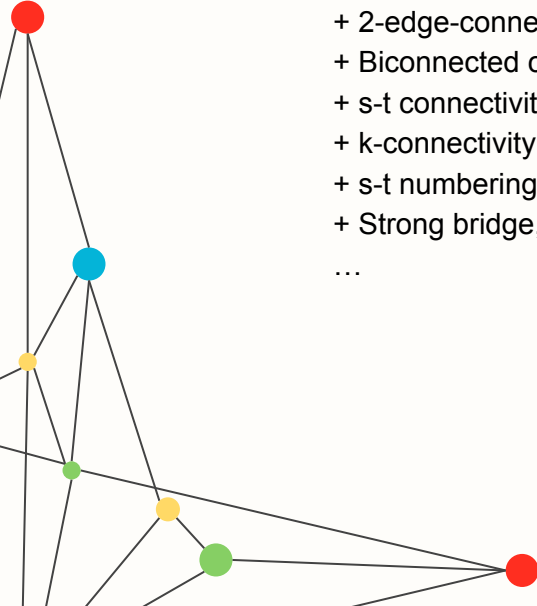
The why?





Algorithmic Applications

Gives insights to other kinds of network's “weakness”

- 
- + 2-edge-connected components
 - + Biconnected components
 - + s-t connectivity
 - + k-connectivity
 - + s-t numbering
 - + Strong bridge, strong articulation points

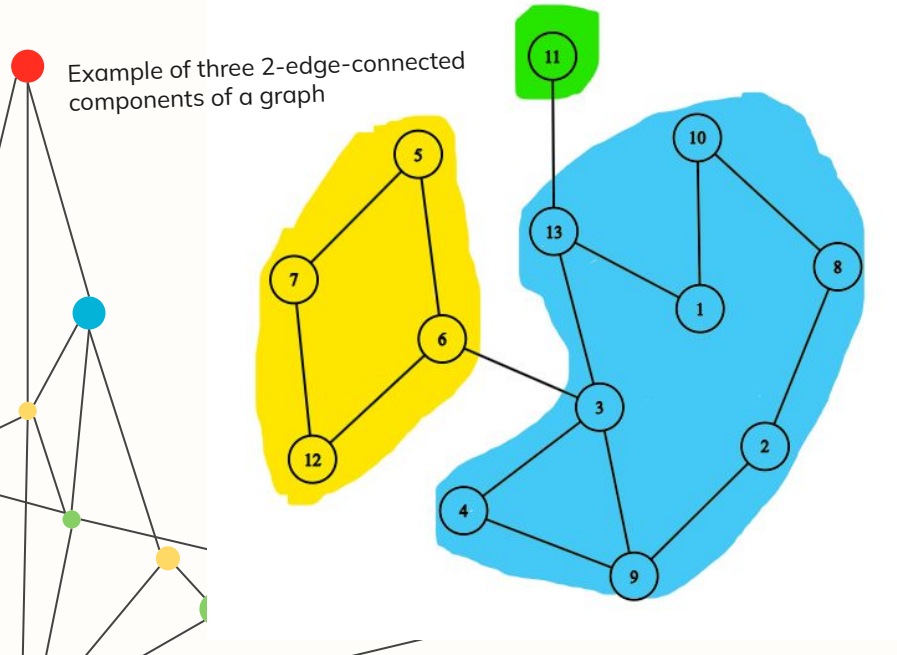
...



Algorithmic Applications

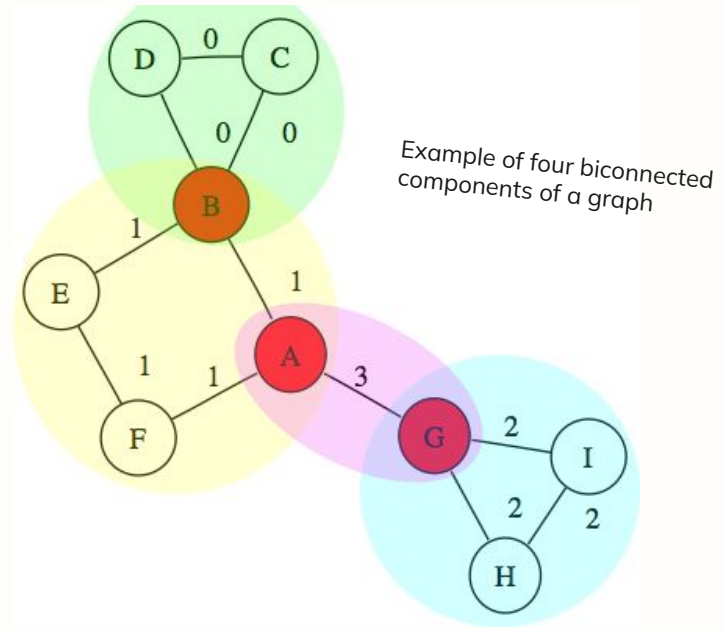
2-edge-connected Component

A maximal connected subgraph with no bridge



Biconnected component

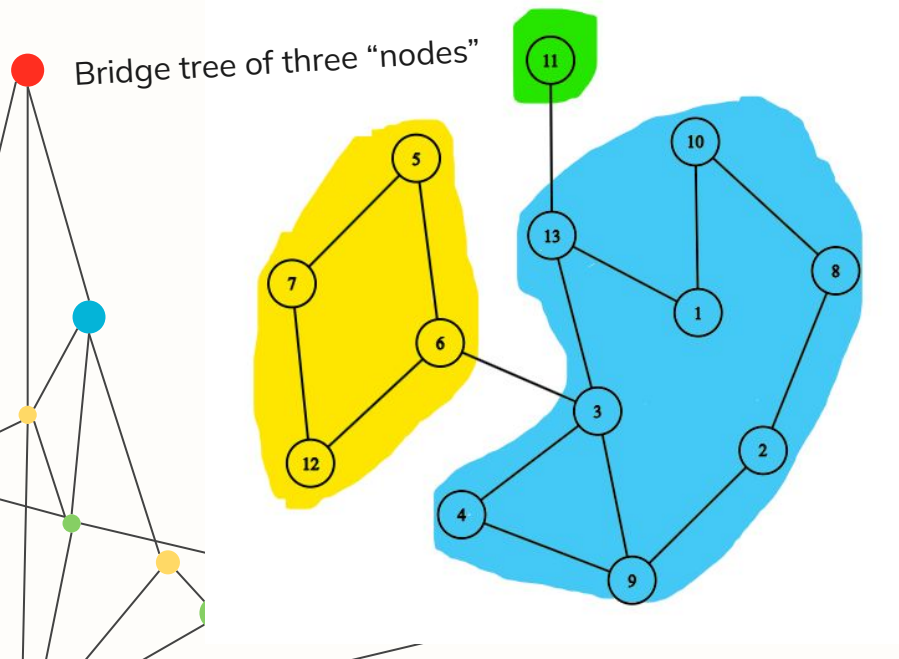
A maximal connected subgraph with no articulation point



Bridge tree

The “tree” that connects the 2-edge-connected components.

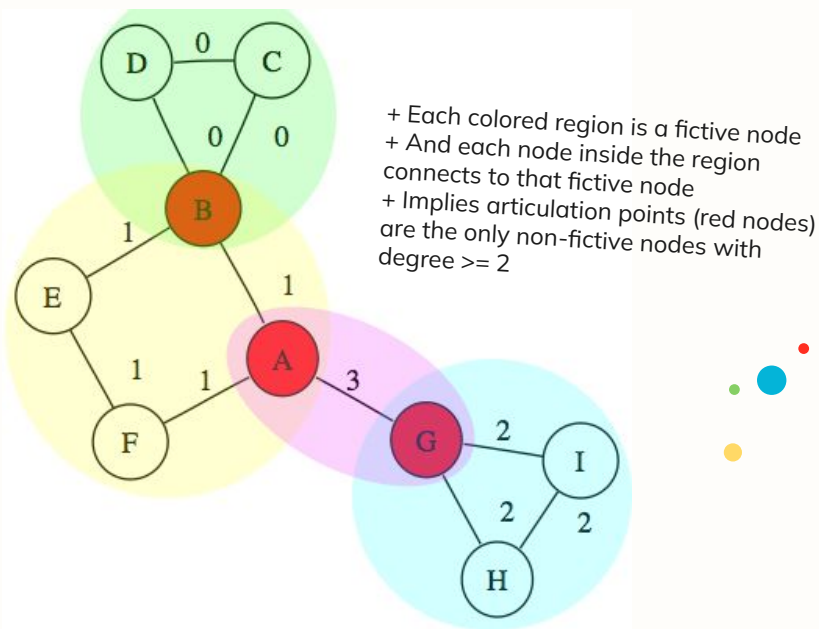
A kind of data structure that helps with online retrieving of connectivity information such as finding critical edges between a specified pair of nodes



Block-cut tree

The bipartite tree where each biconnected component is a *node* on one side and the nodes in that component is on the other side connecting to that *node*

Helps with online retrieving connectivity information such as finding critical vertices between some two nodes.

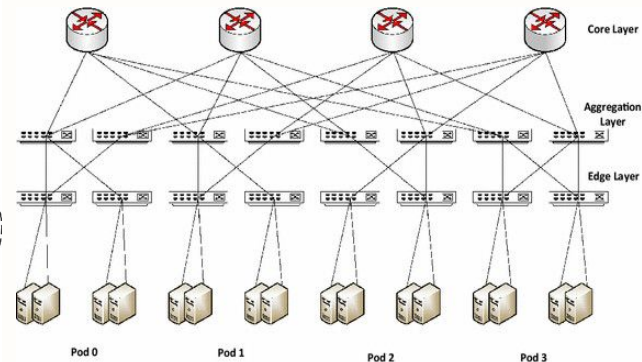
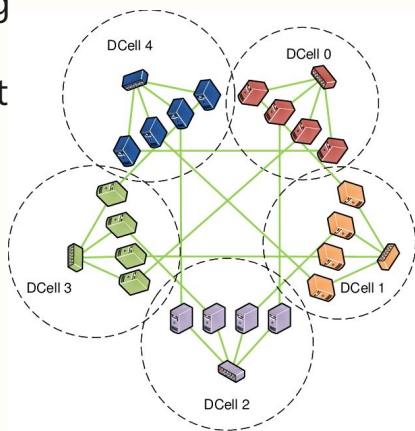
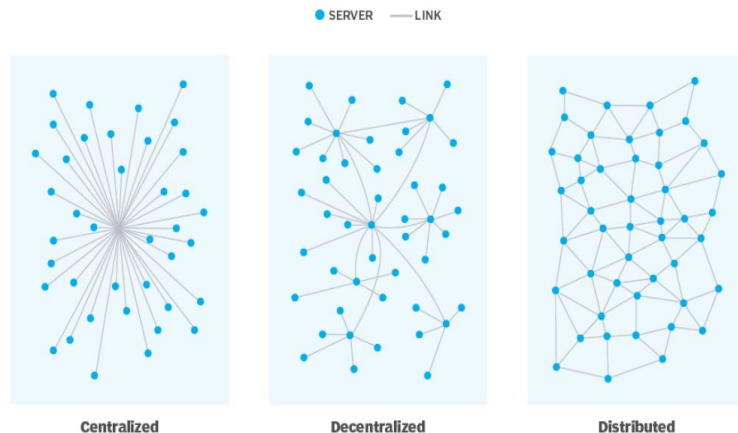


Real-life Applications

[6] [7] [8] [9]

Graph connectivity is centric in network reliability designs.

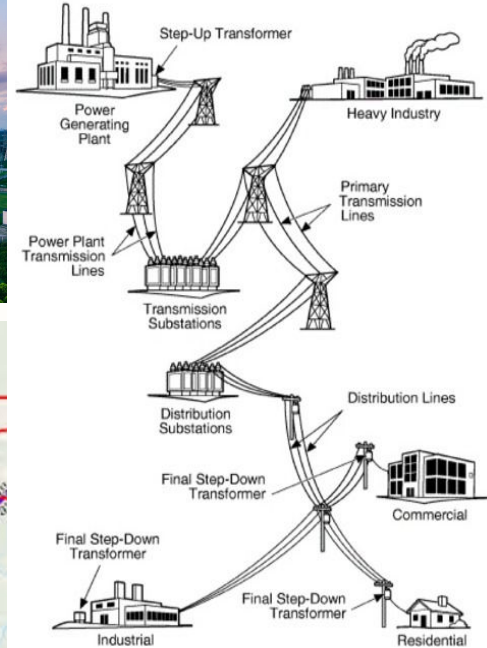
Most cloud provider (AWS, MS Azure...) design their data center networks using topologies that inherently address the concerns of network reliability and fault tolerance.



Real-life Applications

Beyond computer network, network reliability and fault tolerance designs are critical in other area such as Telecommunications, Power Grids, and Transportation Networks.

Understanding the structures' weakness helps with designing an effective roads design, microgrid design and distributed energy systems.



Real-life Applications

In system biology, particularly within protein-protein interaction (PPI) networks, small molecules (SimpleEntity) can be modeled as articulation points.

Small molecules play essential roles in biological systems, e.g. they include metabolites participating as reactants, catalyzers, or inhibitors of reactions. For example, adenosine triphosphate (ATP) and guanosine triphosphate (GTP) are essential metabolites needed as energy sources. [10]

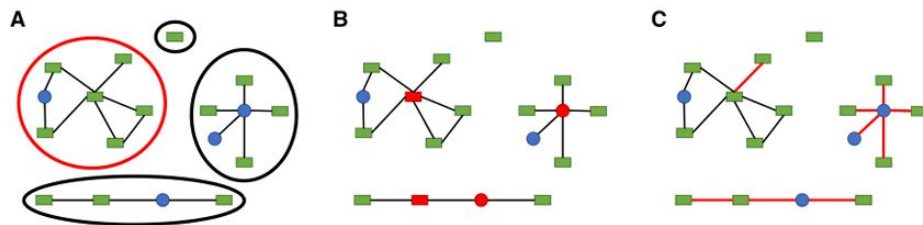
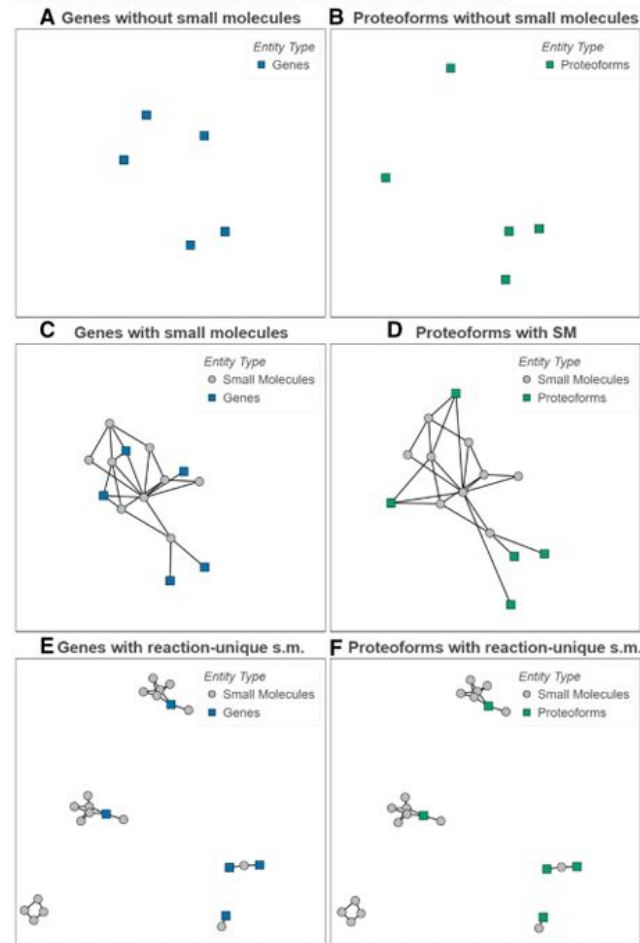
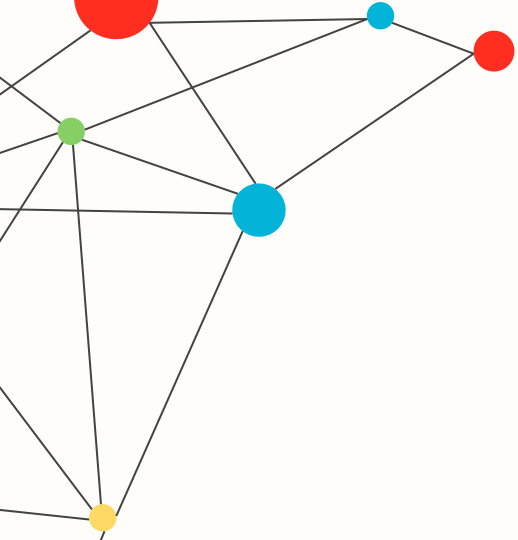


Figure 2. Illustration of graph theory concepts using hypothetical networks with proteoform nodes (green rectangles) and small molecule nodes (blue circles). (A) Connected components of the network, each one surrounded with a dotted line. Largest connected component highlighted with red dotted line. (B) Articulation points, nodes highlighted in red. (C) Bridges, connections highlighted with red lines.

Vitamin B1 (thiamin) metabolism (R-HSA-196819)

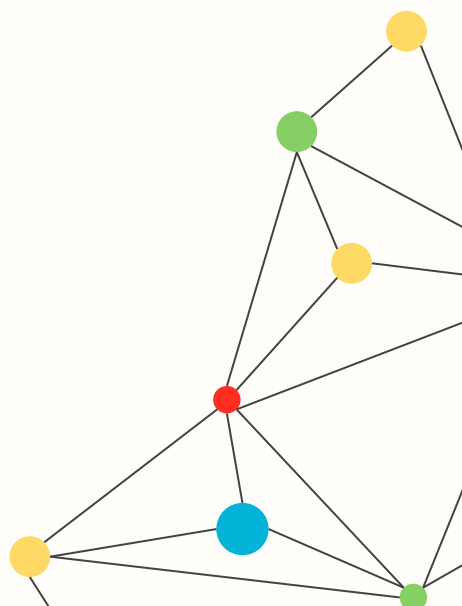


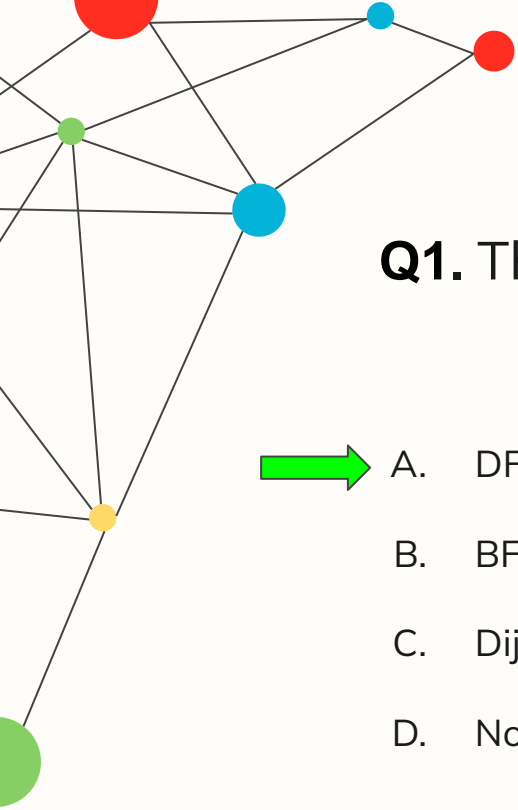


06

Exercises

The fun!

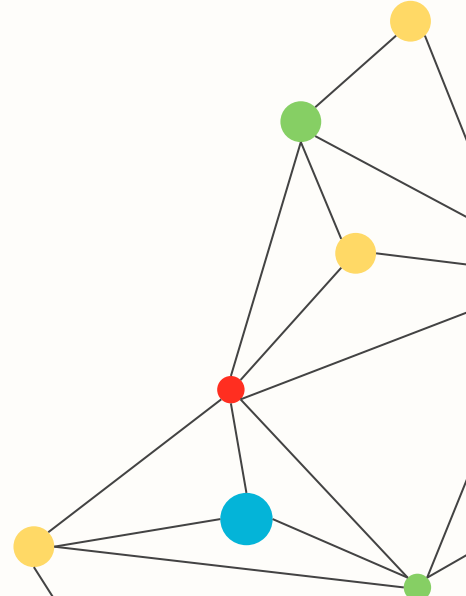


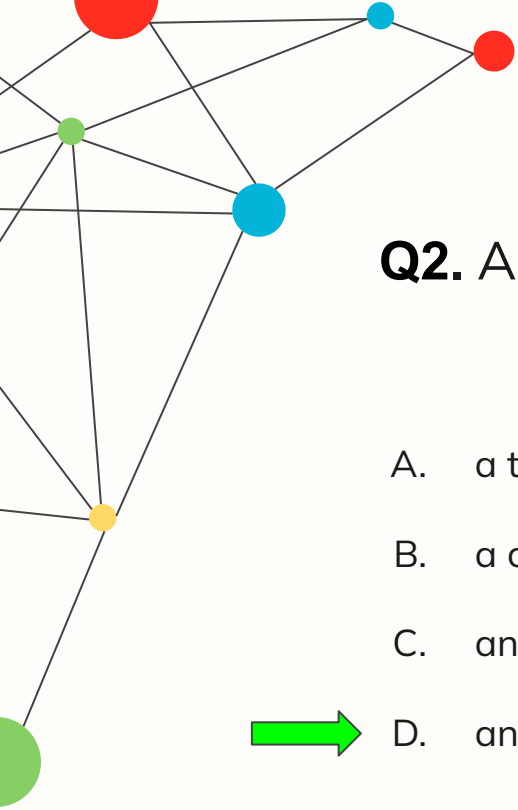


Q1. The Tarjan's algorithm to find bridge uses:



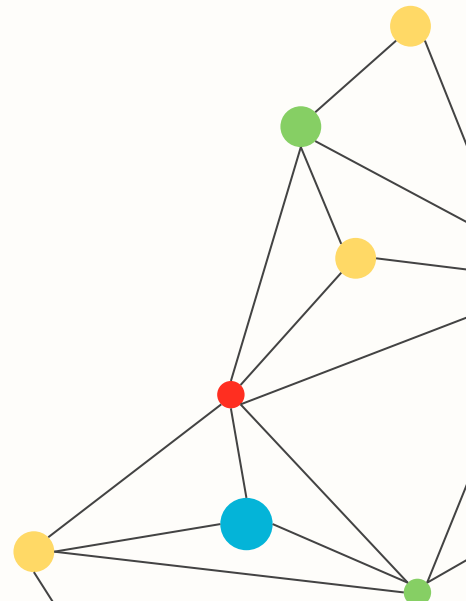
- A. DFS Tree
- B. BFS Tree
- C. Dijkstra Tree
- D. Not given

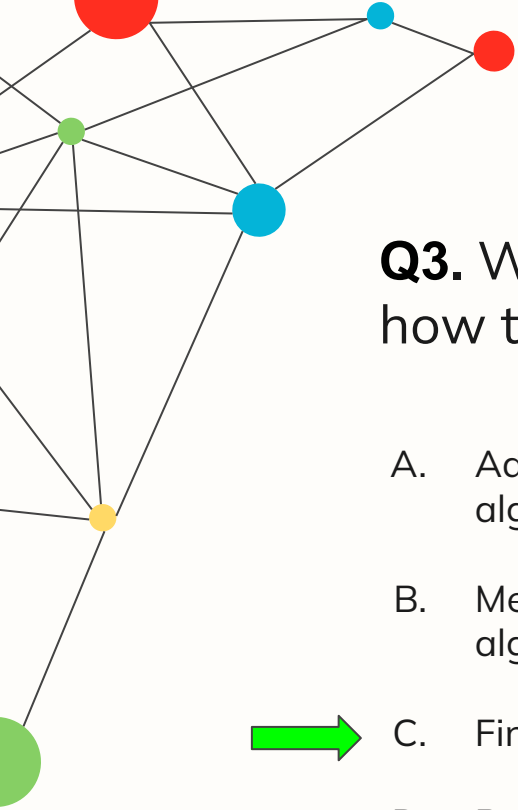




Q2. A bridge is:

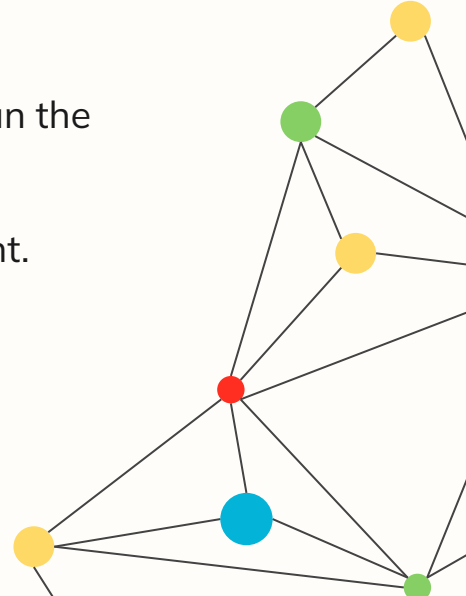
- A. a tree-edge that connects a parent and a child
- B. a cotree edge that goes to the root
- C. an edge connecting two connected components
- ➡ D. an edge that doesn't belong to any cycle

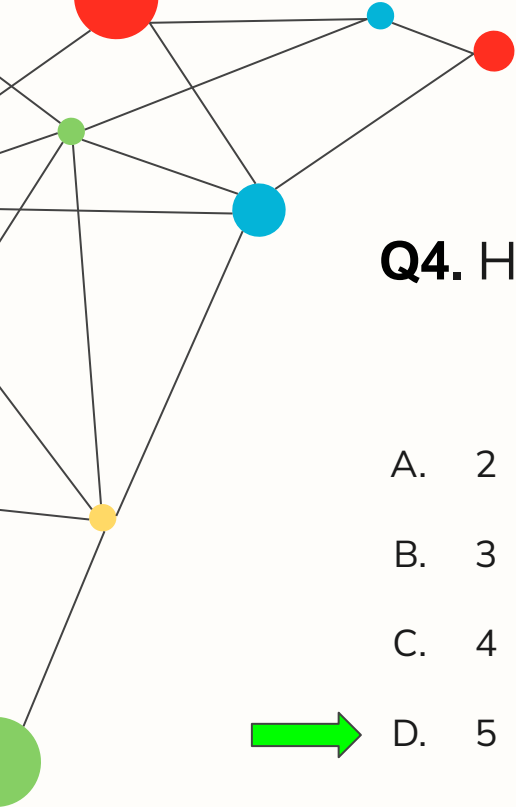




Q3. When dealing with disconnected graph, how to find the articulation points?

- A. Add edges so that the graph become connected and run the algorithm.
- B. Merge nodes so that the graph become connected and run the algorithm.
- ➡ C. Find the articulation points for each connected component.
- D. Run the algorithm rooted at 1.





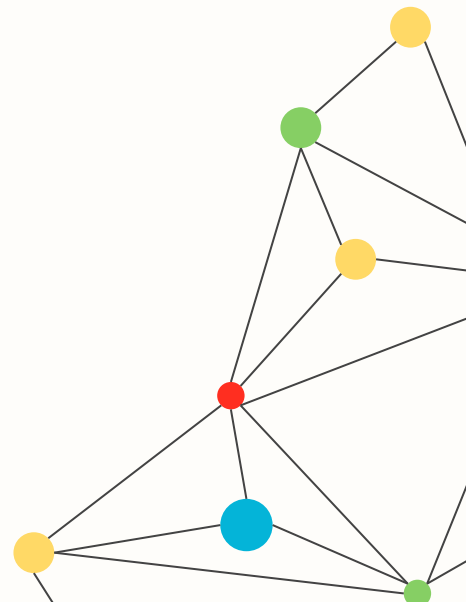
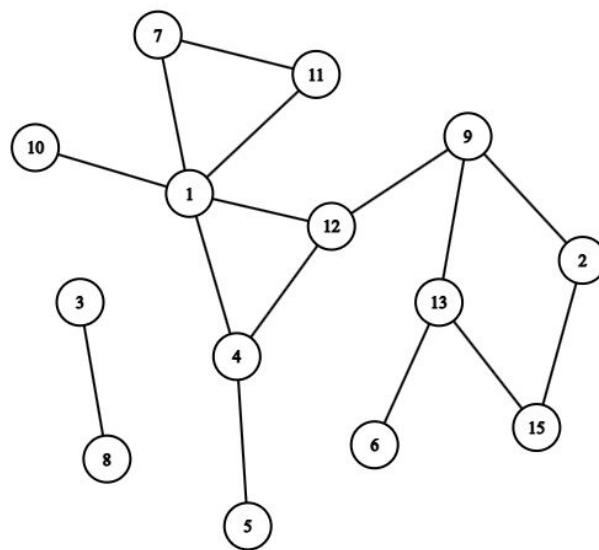
Q4. How many bridges in the graph below?

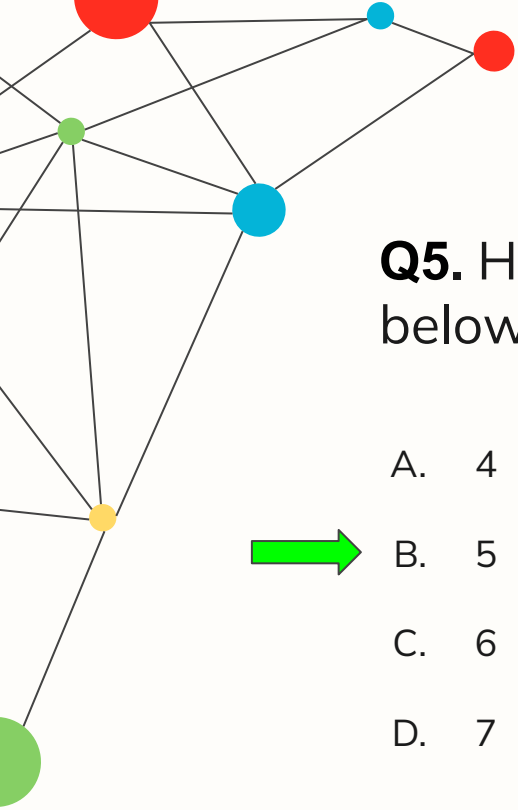
A. 2

B. 3

C. 4

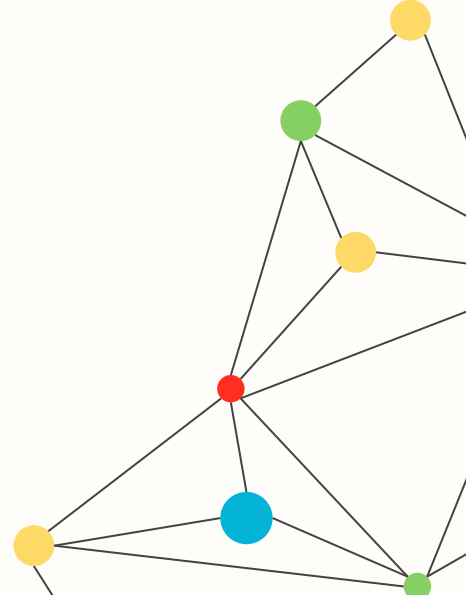
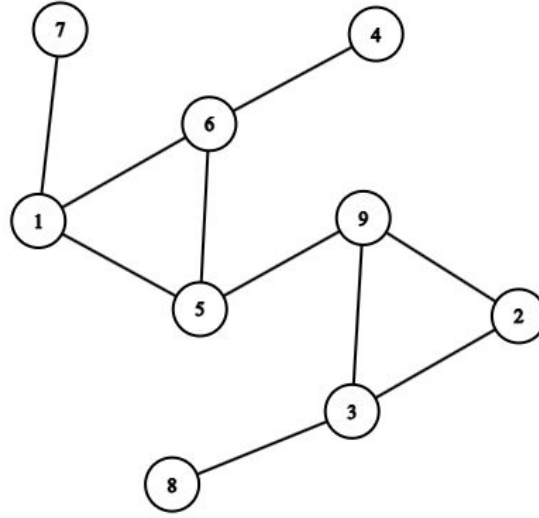
➡ D. 5

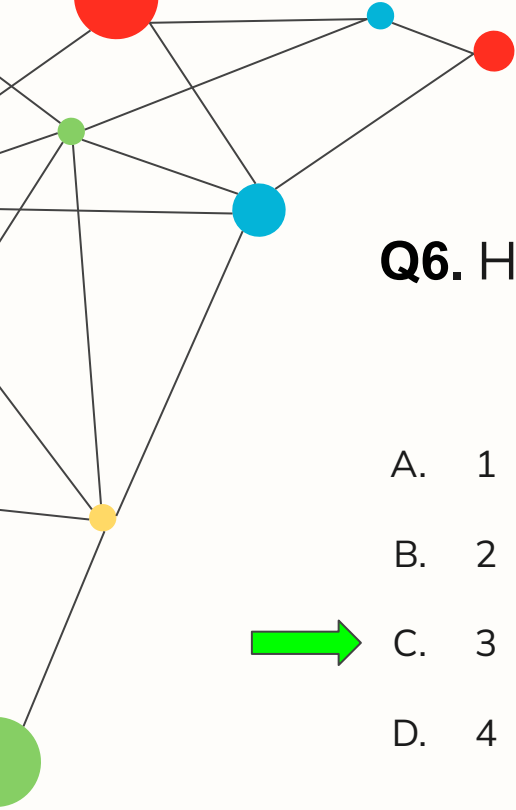




Q5. How many articulation points in the graph below?

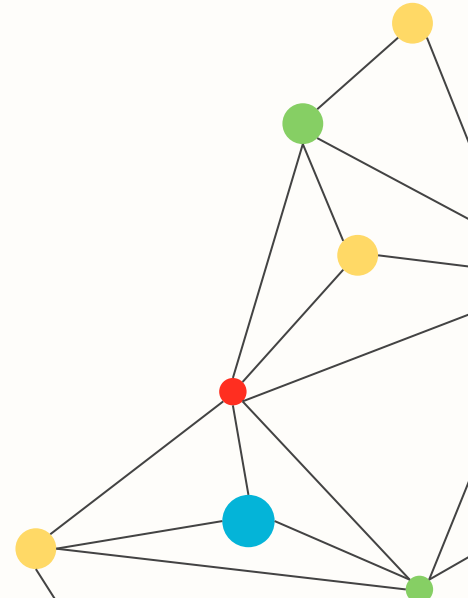
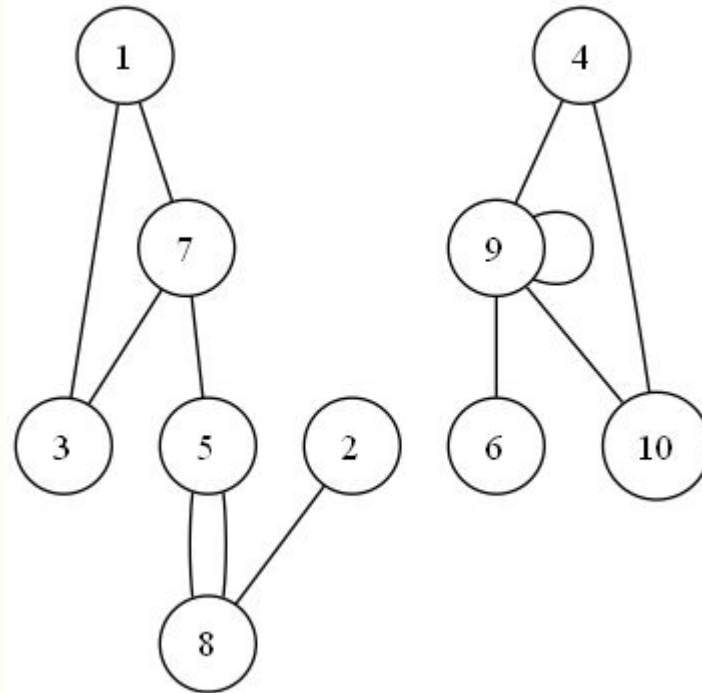
- A. 4
B. 5
C. 6
D. 7
- 

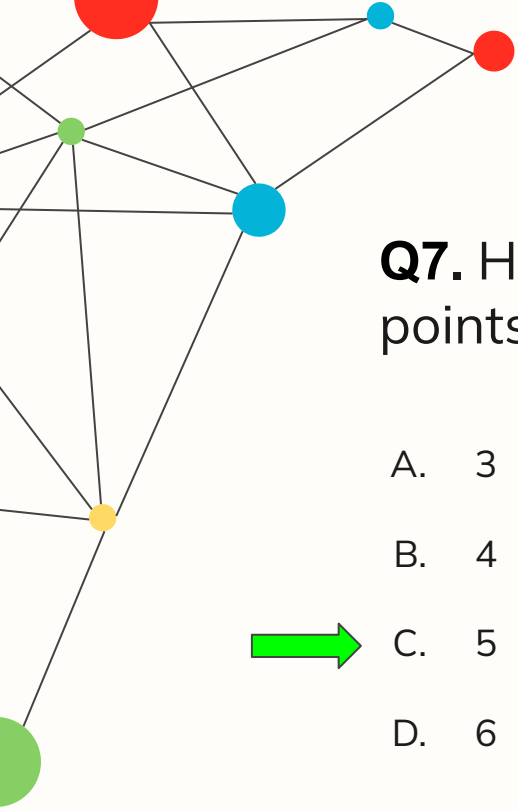




Q6. How many bridges in the graph below?

- A. 1
- B. 2
-  C. 3
- D. 4





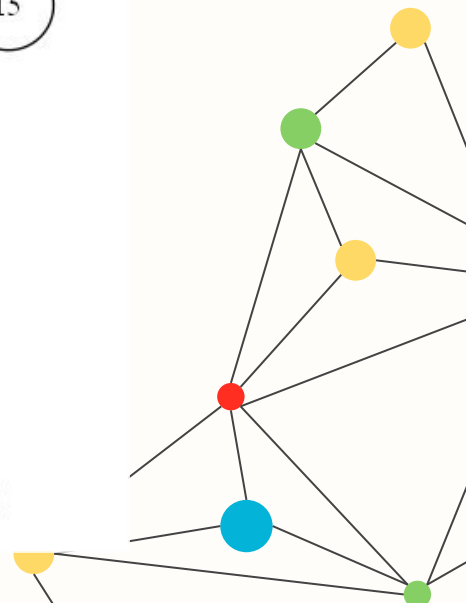
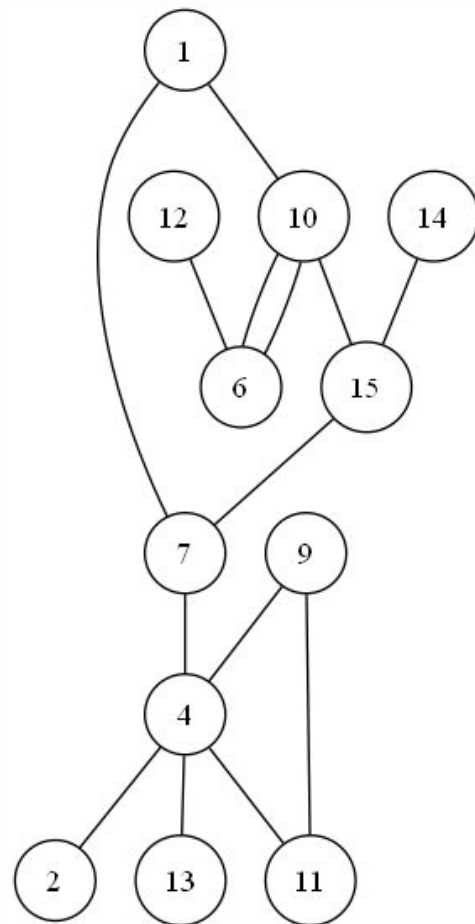
Q7. How many articulation points in the graph?

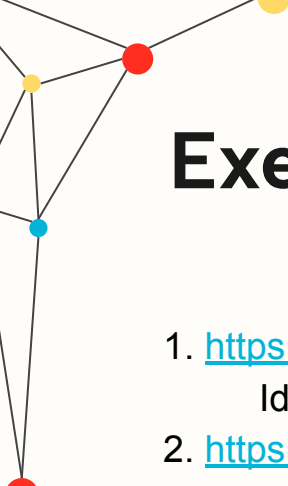
A. 3

B. 4

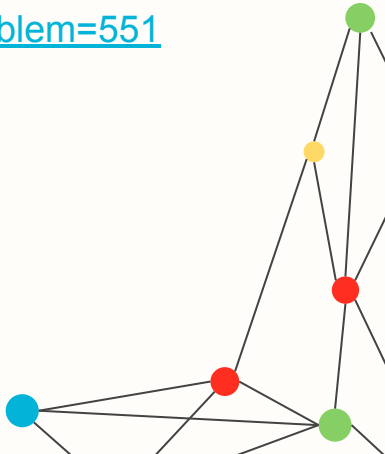
 C. 5

D. 6





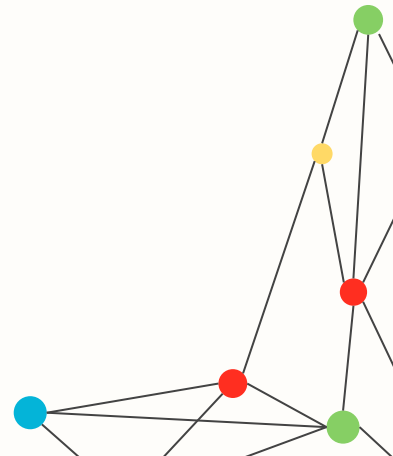
Exercises

1. https://onlinejudge.org/index.php?option=onlinejudge&page=show_problem&problem=737
Identify the bridges
 2. <https://www.spoj.com/problems/SUBMERGE/>
Identify the articulation points
 3. <https://szkopul.edu.pl/problemset/problem/eDt8w290owtatmCjad0O0ywk/site/?key=statement>
Count the number of pairs that get disconnected for each node's failure
 4. https://onlinejudge.org/index.php?option=onlinejudge&page=show_problem&problem=551
Direct the streets (edges) for connected transportation
 5. https://judge.yosupo.jp/problem/two_edge_connected_components
Identify the 2-edge-connected components
 6. https://judge.yosupo.jp/problem/biconnected_components
Identify the biconnected components
- 



Solution

Problem 1 and 2 are immediate practice problems for the concepts introduced in this seminar and are solved by the presented algorithms.





Solution

To solve problem 3, one notices that only articulation points' removals cause disconnection between pair.

In that case when articulation point u is removed, a disconnected pair must have one vertex in one subtree with respect to u and the other in another subtree.

This can be counted by simple combinatorial techniques: $\sum s_i s_j = (\sum s_i)^2 - \sum s_i^2$ (by Principle of Inclusion-Exclusion), where s_i is the size of a subtree connected to u and $i \neq j$.



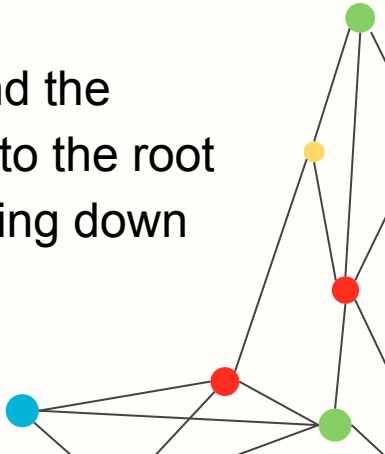


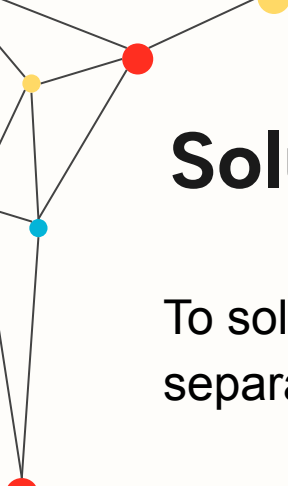
Solution

To solve problem 4, one notices that the graph must not have any bridge for an orientation to be possible.

When there is no bridge, then in the DFS tree, any edge has a cotree edge passing over it.

Thanks to cotree edges, if we direct the tree edges downward and the cotree edges upward towards the root, then any node can travel to the root (by going up the cotree edges) and go to any other nodes (by going down the tree edges).



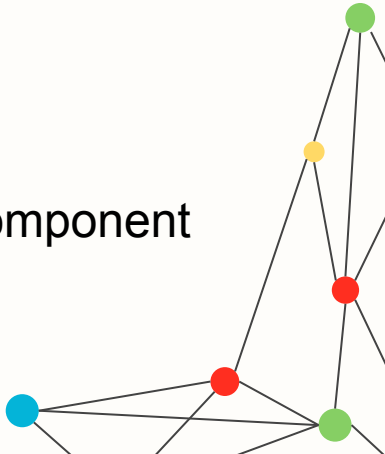


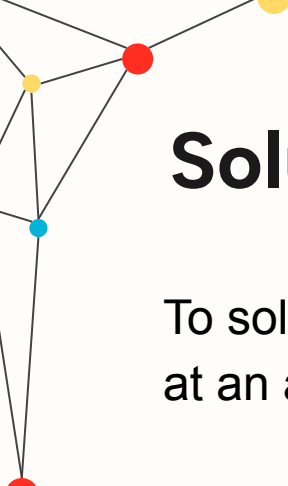
Solution

To solve problem 5, recall that the 2-edge-connected components are separated by the bridges of a graph.

So if, during the DFS process, we push nodes into a stack (or any other container) in post-order of the DFS, then whenever we find a bridge we can identify the set of nodes contained in the stack as one 2-edge-connected component.

(whenever that happens, we should empty the stack for future component identifying)



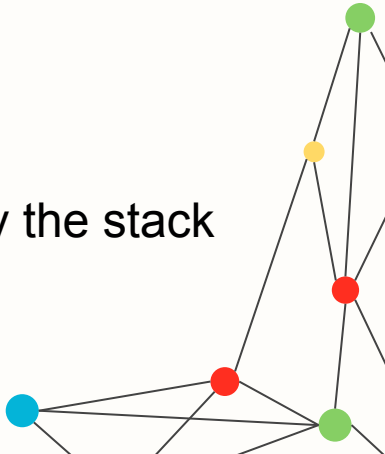


Solution

To solve problem 6, notice that two biconnected components only intersect at an articulation points.

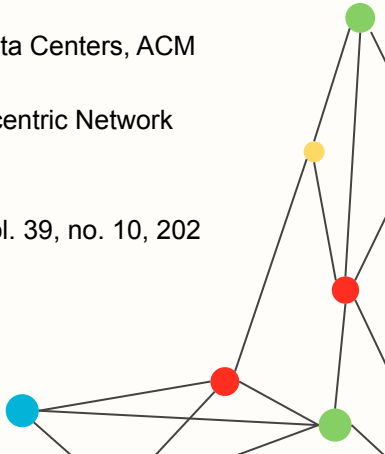
Similarly to problem 5, during the DFS process, if we push the nodes into a stack in post-order then whenever an articulation point is identified, the set of nodes in the stack, along with the articulation point, forms one biconnected component.

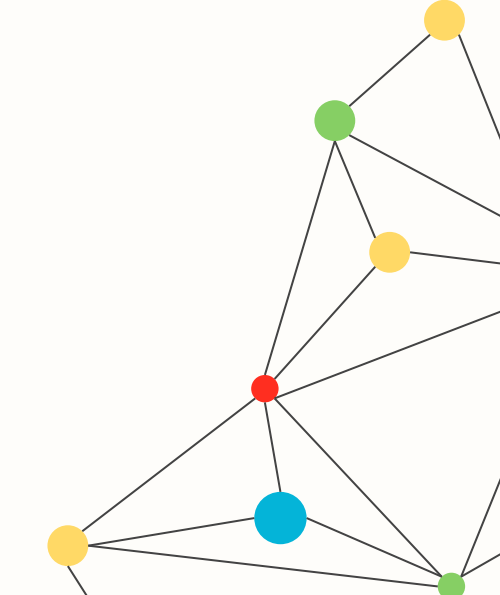
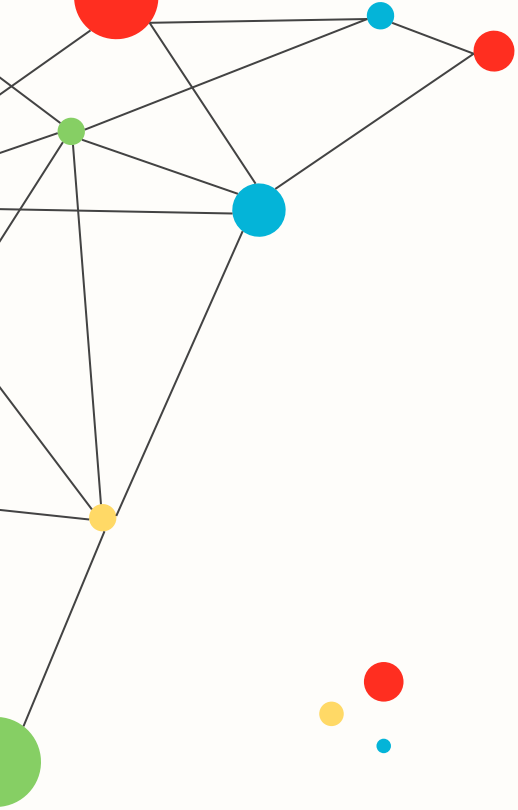
When one biconnected component is identified, we should empty the stack and leave only the articulation point remained.





Reference

- [1] Tarjan, R. (1972). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, 1(2), 146–160.
 - [2] R. E. Tarjan, "A Note on Finding the Bridges of a Graph," *Information Processing Letters*, vol. 2, no.6, pp. 160–161, 1974.
 - [3] Hopcroft, J., Tarjan, R. (1973). Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6), 372–378.
 - [4] G. F. Italiano and E. Nardelli, "Finding Strong Bridges and Strong Articulation Points in Linear Time," *Theoretical Computer Science*, vol. 447, pp. 55–61, 2012.
 - [5] Online Bridge Finding (CP-Algorithm)
 - [6] A. Cornejo, S. Gilbert, and N. Lynch, *Fault-Tolerance Through k-Connectivity*, MIT CSAIL Technical Report, 2010.
 - [7] M. Al-Fares, A. Loukissas, and A. Vahdat, A Scalable, Commodity Data Center Network Architecture, *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74, 2008.
 - [8] C. Guo, H. Wu, K. Tan, L. Shi, Y. Zhang, and S. Lu, DCell: A Scalable and Fault-Tolerant Network Structure for Data Centers, *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 75–86, 2008.
 - [9] C. Guo, G. Lu, D. Li, H. Wu, X. Zhang, Y. Shi, C. Tian, Y. Zhang, and S. Lu, BCube: A High Performance, Server-centric Network Architecture for Modular Data Centers, *ACM SIGCOMM Computer Communication Review*, vol. 39, no. 4, pp. 63–74, 2009.
 - [10] J. Smith et al., Extending protein interaction networks using proteoforms and small molecules, *Bioinformatics*, vol. 39, no. 10, 202
- 



Thanks!

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)